

Distributed Point Function with Constraints, Revisited*

Keyu Ji[†] Bingsheng Zhang[†] Hong-Sheng Zhou[‡] Kui Ren[†]

[†]Zhejiang University. Email: {jikeyu, bingsheng, kuiren}@zju.edu.cn.

[‡]Virginia Commonwealth University. Email: hszhou@vcu.edu.

Abstract

Distributed Point Function (DPF) provides a way for a dealer to split a point function $f_{\alpha,\beta}$ into multiple succinctly described function-shares, where the function $f_{\alpha,\beta}$ for a special input α , returns a special output value β , and returns a fixed value 0 otherwise. As the security requirement, any strict subset of the function-shares reveals nothing about the function $f_{\alpha,\beta}$. However, each function-share can be individually evaluated on the common input x , and these evaluation results can then be merged together to reconstruct the value $f_{\alpha,\beta}(x)$.

Recently, Servan-Schreiber *et al.* (S&P 2023) investigate the access control problem for DPF; namely, the DPF evaluators can ensure that the DPF dealer is authorized to share the given function with privacy assurance. In this work, we revisit this problem, introducing a new notion called *DPF with constraints*; meanwhile, we identify that there exists a subtle flaw in their privacy definition as well as a soundness issue in one of their proposed schemes due to the lack of validation of the special output value β . Next, we show how to reduce both the storage size of the constraint representation and the server’s computational overhead from $O(N)$ to $O(\log N)$, where N is the number of authorized function sets. In addition, we show how to achieve fine-grained private access control, that is, the wildcard-style constraint for the choice of the special output β . Our benchmarks show that the amortized running time of our logarithmic storage scheme is $2\times - 3\times$ faster than the state-of-the-art when $N = 2^{15}$. Furthermore, we provide the first impossibility and feasibility results of the *DPF with constraints* where the evaluators do not need to communicate with each other.

*This is an updated version of our work “Fine-grained Policy Constraints for Distributed Point Function”, which can be found on IACR ePrint Report 2023/1672; please see [21]. Compared with [21], this version has been significantly improved. We revised and improved the definition of *DPF with constraints*; we added new impossibility and feasibility results of the DPF with constraints where the evaluators do not need to communicate with each other; finally, to improve the presentation and make the results more readable, we removed the part of the DPF with attribute-based constraints.

Contents

1	Introduction	1
1.1	Our contributions	2
1.2	Related work	4
1.3	Organization	5
2	Preliminaries	5
3	DPF with Constraints	6
4	Revisiting VDPF-PACL in [33]	8
5	CDPF with Logarithmic Storage	10
5.1	Incremental VDPF	10
5.2	Our construction using incremental VDPF	12
6	CDPF with Wildcard-style Constraint	15
6.1	Efficiency	16
7	Implementation and Benchmark	16
7.1	Proof time	17
7.2	Communication overhead	17
7.3	Verification cost	18
8	CDPF with Non-interactive Evaluators: Impossibility and Feasibility	19
A	Definition of VDPF	22
B	Definition of PACL	23
C	Properties of Definiton 1 for the Special Case	25
D	CDPF with Non-Interactive Evaluators in the Preprocessing Model	25
E	Security Proofs	28
E.1	Proof of Theorem 1	28
E.2	Proof of Theorem 2	29
E.3	Proof of Theorem 3	33

1 Introduction

Function Secret Sharing (FSS) is initially introduced by Boyle *et al.* [6]. In FSS, a dealer is allowed to split a secret function f into multiple *succinctly described* function-shares $\{f^{(i)}\}_{i \in \mathbb{Z}_n}$, and any strict subset of the function-shares reveals nothing about the function f . Here, each function-share $f^{(i)}$ can be *individually* evaluated on the common input x , and these evaluation results $\{f^{(i)}(x)\}_{i \in \mathbb{Z}_n}$ can then be *merged together* so that the value $f(x)$ is reconstructed.

Distributed point function. Distributed Point Function (DPF) [15] is arguably the most widely used special case of FSS, where the secret function f is a *point function*. Concretely, a point function f can be defined as follows:

$$f(x) = \begin{cases} \beta & \text{if } x = \alpha \\ 0 & \text{otherwise} \end{cases}$$

We refer to α and β as the *special input* and *special output* of the function f , respectively. DPF is at the core of many privacy-preserving systems, such as distributed ORAM [8, 13, 20], anonymous communication [9, 14, 28, 34], private telemetry [3, 26], privacy-preserving machine learning [5, 18, 25, 30], and payment system [29].

Distributed point function with constraints. In the above applications, DPFs are used to encode and process *private* user data, often in settings where users may be malicious. Authentication mechanism for DPF therefore becomes an important problem in many real-world situations. We highlight three representative scenarios.

- *Anonymous authentication in distributed systems.* DPF is used to privacy-preserving updates in distributed ledgers and accounting services [29]. Specifically, in a given transaction, the special input of DPF encodes the initiator’s (or the target) accounts, while the special output represents a monetary amount. Without anonymous authentication, an adversary can forge DPF shares encoding unauthorized transfers, easily stealing funds.
- *Spam prevention in anonymous communication systems.* Leveraging efficient DPF construction, a variety of anonymous communication systems achieve practical performance [9, 14, 28, 34]. We note that *sender-anonymous messaging* is a core functionality in such systems, which protects whistleblowers and other sensitive users. However, sender anonymity also makes such systems particularly susceptible to denial-of-service attacks (e.g., malformed writes or Sybil flooding). To prevent spam, it requires access control method for DPFs that encoding the messages [14, 28, 34].
- *Access control in private databases.* Multi-server PIR and private databases realized via DPFs allow users to efficiently query large datasets without revealing which items they access [8, 13, 20]. In such systems, access policies should restrict which records a given user may query or update. For instance, in e-health databases, records are owned by different parties; private access control is therefore necessary to prevent malicious users from accessing or modifying unauthorized records.

While a standard DPF technique protect the privacy of the the special input α and special output β , it *cannot restrict* which point functions a dealer is allowed to share. This motivates the study of *DPF with constraints*. Informally, a *DPF with constraints* enables the data owner to specifies, for each user, a collection of authorized point functions (i.e., an access structure over (α, β) pairs).

Example application: Using CDPF for private database access control. In Fig. 1 below, we illustrate the system overview of private databases with access control based on *DPF with constraints*. The system involves the following entities: (1) the data owner who outsources (encrypted) data to servers and determines the constraint policy, (2) the user (i.e., the DPF dealer) who secretly shares a function to access the database and proves it satisfies the constraint, and (3) the servers (i.e., the DPF evaluators) locally evaluate the function share and obviously verify the user’s access. Note that, the user needs a valid secret key as authorization to pass the verification. There are various techniques for key distribution, e.g., anonymous communication channels. We note that the key distribution mechanism is orthogonal to the constrained DPF. For simplicity, in this paper, we assume there exists a trusted authority to issue the secret keys.

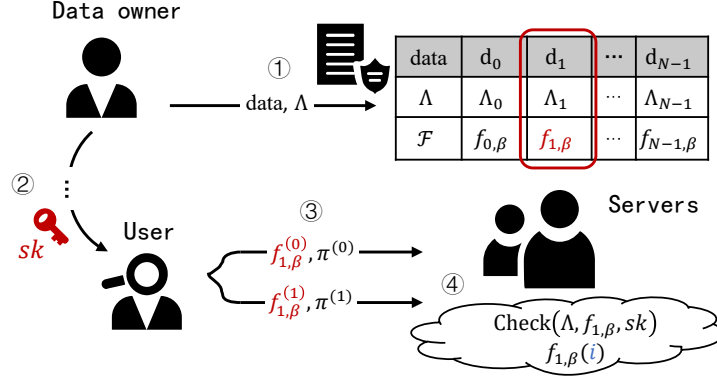


Figure 1: System overview of private databases with access control based on constrained DPF: ① the data owner outsources (encrypted) data with constraint configurations to servers; ② distribute secret keys; ③ the user holds the secret key for $f_{1,\beta}$ accesses d_1 by distributing secret shares of $f_{1,\beta}$ and proof shares to servers; ④ the servers jointly check whether this access satisfies the constraint and evaluate $f_{1,\beta}(i)$ on any common input i .

Initial results by Servan-Schreiber et al.’s, and security concerns. Very recently, Servan-Schreiber *et al.* [33]¹ initiate the study on *DPF with constraints*, which they call “Private Access Control Lists (PACLs) for DPF.” Their most efficient construction, named VDPF-PACL, is based on verifiable DPF [3, 12]. The verifiable DPF is a special type of DPF that additionally allows evaluators to check whether the function shares are well-formed. Servan-Schreiber *et al.* aim to leverage this property to prevent a malicious dealer from “selecting” an invalid constraint configuration. Unfortunately, we find that this scheme has a soundness issue due to the lack of assessment of the function output.

Achieving fine-grained access control. On the other hand, the data owner might require more fine-grained access control than those supported in [32, 33]. Recall Fig. 1, the system may be for a constrained writing scenario: a landlord (i.e., the data owner) provides a template lease as a data item; the tenant (i.e., the user) is only allowed to modify the lease term within the template, and all other terms are fixed. In such scenarios, the data owner needs to constrain the user’s choice of the special output β , which represents the value to be written. More specifically, to obviously write to location α of a database $D := \{d_i\}$, the user secret-shares $f_{\alpha,\beta}$ with the servers. The servers then jointly update the database, setting each item $d_i := d_i + f_{\alpha,\beta}(i)$. Note that here only the α -th item is modified to a new value $d_\alpha + \beta$. To achieve the constrained writing, the servers need to verify the value of β before updating the database. Unfortunately, none of schemes in [32, 33] supports DPF constraints w.r.t. β .

Achieving better performance. In addition to enabling fine-grained private database access control, all previous approaches require that each evaluator stores at least N verification keys $\{\mathbf{vk}_{x_i}\}_{i \in \mathbb{Z}_N}$ for N constraint configurations assigned for distinct input values $\{x_i \in \mathcal{D}\}_{i \in \mathbb{Z}_N}$. Typically, the evaluators jointly compute $\{f_{\alpha,\beta}(x_i)\}_{i \in \mathbb{Z}_N}$ in the shared form, which is then used to perform the 1-out-of- N key selection to obtain $\mathbf{vk}_\alpha$. This results in $O(N)$ key storage and $O(N)$ key selection computation complexity. We observe that such selection step becomes a performance bottleneck of CDPF when N is large. We are wondering whether there exists a more efficient solution for the key selection.

In this work, we focus on 2-party *DPF with constraints*, or *constrained DPF* (CDPF). To the best of our knowledge, the existing multi-party DPF systems are less efficient [6, 9, 28].

1.1 Our contributions

In this paper, we systematically investigate the problem of DPF with constraints. Our contributions can be summarized as follows.

¹Note, the updated version of [33] can be found on IACR ePrint; please see [32].

New definitions. We propose formal definitions for DPF with constraints (CDPF), which can be viewed as an extension of verifiable DPF [12]. For a verifiable DPF, the evaluators can only ensure the shared function is a valid point function; whereas, in CDPF, the evaluators can also obviously verify whether the dealer is authorized to share this point function.

In addition, the PACLs for DPF in [33] can be viewed as a CDPF. However, the privacy definition in [33] is problematic in several aspects. Firstly, their definition did not capture the rushing adversary model; namely, the simulator is given the adversary’s message as input. Furthermore, some algorithm of their CDPF scheme takes the function shares generated by DPF as its input, and outputs evaluation results and audit tokens for verification. However, in the privacy definition in [33], the view of evaluator does not include the function share. It results in a trivial simulator (cf., the proof of Theorem 3 in [33]) who simulates the audit token independent of function share. In Appendix B, we provide a simple counterexample to illustrate this subtle problem.

Breaking and fixing the scheme in [33]. We find that the efficient CDPF scheme (i.e., VDPF-PACL) in [33] is insecure against a malicious dealer. More specifically, consider the group $\mathbb{G} := \mathbb{Z}_p^*$ with generator g , and a function family $\mathcal{F}$ of point functions $f : \mathcal{D} \rightarrow \mathbb{Z}_p$. For each $x \in \mathcal{D}$, VDPF-PACL associates a verification key $\text{vk}_x := g^{\text{sk}_x}$ as the constraint configuration, intended to authorize point functions $\{f_{x,v}\}_{v \in \mathbb{Z}_p}$, where the data owner holds the secret key sk_x . Upon receiving function-shares $\{f_{\alpha,\beta}^{(i)}\}_{i \in \mathbb{Z}_n}$ from dealer, for the authorization verification, each evaluator P_i computes $\text{vk}^{(i)} := \sum_{x \in \mathcal{D}} f_{\alpha,\beta}^{(i)}(x) \cdot \text{vk}_x$ to select the corresponding constraint configuration. VDPF-PACL assumes $\beta = 1$ and enables evaluators to check whether the dealer knows $\log_g(\text{vk}_\alpha \cdot \beta)$. Although authors in [33] claim that any user without knowledge of sk_α cannot pass the verification. Unfortunately, their scheme never validates $\beta = 1$; the malicious dealer can bypass the verification by setting $\beta := g^r / \text{vk}_\alpha$ for any forged secret key $\text{sk}_\alpha^* := r$. We provide a formal description of our attack in Appendix 4.

To fix this issue, a straightforward way is to fix β to some public value, say 1, and validate $\beta = 1$ during the verification phase. Yet, sometimes the dealer needs to set β to a specific private value that is unknown to the evaluators. For instance, in a secure mailbox system [14], the user sends a private message to a specific mailbox by secret-sharing a point function $f_{\alpha,\beta}$, where α represents the mailbox address and the value of β is set to the content of its private message.

In this work, we propose an alternative solution using verifiable DPF with auxiliary output. The auxiliary output is independent of the original (secret) point function, thus it can be pre-fixed regardless the value of β . Concretely, given a point function $f_{\alpha,\beta} : \mathcal{D} \rightarrow \mathbb{Z}_p$, we transform it to a new point function $f_{\alpha,(\beta,1)} : \mathcal{D} \rightarrow \mathbb{Z}_p \times \mathbb{Z}_p$ as follows:

$$f_{\alpha,(\beta,1)}(x) = \begin{cases} (\beta, 1) & \text{if } x = \alpha \\ (0, 0) & \text{otherwise} \end{cases}$$

For each input $x \in \mathcal{D}$, the first coordinate of the new function output is equal to the output of the original $f_{\alpha,\beta}$, and the second coordinate is either 0 or 1 which can be used to defend against our attack on the VDPF-PACL scheme in [33]. In addition, adding an auxiliary output only results in the communication overhead of transmitting a correction word and the computation overhead of a few addition operations. In another word, our fix introduces little overhead.

Improving the performance of the schemes in [32, 33]. We show how to improve the performance of the CDPF schemes in [32, 33]. Let N be the number of constraint configurations. We can achieve $O(\log N)$ storage size and $O(\log N)$ selection computation complexity by packing the configurations. Recall that, in [32, 33], for a family of point functions with input domain $\mathbb{Z}_N$, each input $i \in \mathbb{Z}_N$ is mapped to a random key pair $(\text{vk}_i, \text{sk}_i)$, which yields $O(N)$ storage. Let $i_1, i_2, \dots, i_{\log N}$ be the binary representation of i . Our main idea is to generate $2 \log N$ random keys $((k_{1,0}, k_{1,1}), \dots, (k_{\log N,0}, k_{\log N,1}))$ and map i to $\text{vk}_i := \prod_{j=1}^{\log N} k_{j,i_j}$. Then, 1-out-of- N key selection is transformed into $\log N$ number of 1-out-of-2 selections.

Note that, there are two challenges to this intuition. First, our new selection method utilizes the binary representation of the special input α of the secret-shared point function, but the evaluators cannot obviously bit-decompose α using the (verifiable) DPF technique. To overcome this problem, we introduce a primitive of verifiable incremental DPF (VIDPF) [10, 11, 27]. More specifically, we propose an efficient *batch prefix evaluation* construction for VIDPF, which takes a set of evaluation values $X \subseteq \mathbb{Z}_N$ as input, and outputs a series of prefix evaluation results. If $\alpha \in X$, these prefix evaluation results can form the (scaled) bit

decomposition of α . The second challenge is that if the secret key is exactly the discrete logarithm of the corresponding verification key, then a malicious user with access to multiple indices may be able to compute the valid secret keys of other unauthorized indices. We introduce bilinear map to address this problem, which blinds the discrete logarithm (cf. Sec. 5.2, below).

New CDPF scheme. We propose a new CDPF scheme that supports *wildcard-style constraints* for the special output β . It is useful in the constrained writing setting, such as filling out template contract described above. In our *wildcard-style CDPF* scheme, each constraint configuration is represented by (at least) one constraint string rs , which divides the output of the associated functions into wildcard bits (corresponding to 0 bits in rs) and constrained bits (corresponding to 1 bits in rs). We say a function satisfies the constraint iff all constrained bits of its output are 0; while the wildcard bits can be chosen arbitrarily. That is, the AND result of the function output and the constraint string rs is 0. Our construction is based on the fact that the AND operation is distributive over the XOR operation; therefore, the evaluators can locally compute the XOR secret shares of the AND result between the XOR-shared function output and the constraint string (cf. Sec. 6).

Better performance. Our schemes improve upon the state-of-the-art in terms of both performance and functionality. In Sec. 7, we show that when the number of constraint configurations is large, our VIDPF-CDPF significantly improves the performance compared to the state-of-the-art CDPF schemes (i.e. DPF-PACL and VDPF-PACL) in [32, 33]. For instance, with 2^{15} constraint configurations, the verification speed of our VIDPF-CDPF is $2.5\times$ faster than the DPF-PACL and $2\times$ faster than the VDPF-PACL. In addition, we evaluate our wildcard-style CDPF, and show it can be composed with other constraints to achieve efficient fine-grained access control.

Removing the interaction between CDPF evaluators: Impossibility and feasibility. Notice that all our constructions (as well as all existing constructions) require the evaluators to have (at least) one round of interaction between each other, before the decision is made. A natural question is whether there exists a *non-interactive* CDPF scheme where the evaluators could make a decision without any interaction between each other. In Sec. 8, we demonstrate two results: (1) we show in the *plain model*², it is impossible to have such CDPF schemes; (2) using certain setups, it is feasible to construct such schemes.

For the impossibility result, intuitively, suppose there exists a secure CDPF that supports verification via non-interactive evaluators. That is, the verification result only depends on a single piece of function share. The malicious dealer can invoke the simulator to produce valid function shares for unauthorized functions, which violates the soundness.

For the feasibility result, we show how to construct a non-interactive CDPF scheme in the NIZK and PKI hybrid model, assuming all parties are connected by a broadcast channel. We let the dealer broadcast the encrypted function shares and the audit tokens (needed for the evaluators to make a decision). In addition, the dealer provides an NIZK proof to prove the validity of these audit tokens. Finally, each evaluator can locally check the audit tokens and the NIZK proof to make a verification decision. Furthermore, we propose a more efficient construction in the preprocessing model. Although this scheme has a drawback that it cannot ensure the consensus between evaluators, it is secure enough for PIR applications [19, 35]. Intuitively, the drawback can be addressed by re-randomizing the servers' responses.

1.2 Related work

Recent anonymous communication systems [9, 14, 28, 34] utilize DPF to enable users privately write messages. To prevent private data against malicious users, some of them design ad-hoc methods of access control for DPF. The "mailbox" system Express [14] assigns a λ -bit virtual address to each mailbox, which is regardless of the actual number of mailboxes. Only users who know the secret virtual address can deposit message into the corresponding mailbox. Its long virtual address causes costly overhead for DPF evaluation, and its verification requires interaction between servers and client. Sabre [34] improves Express by a secret-shared non-interactive proof. It achieves less communication and computation costs, but requires an extra aided-server for auditing the well-formedness of DPF. Spectrum [28] constructs an anonymous broadcasting

²The plain model refers to a setting where schemes do not rely on any form of setup, and their security is based on computational hardness assumptions.

system. Its access control mechanism is based on secret-shared Carter-Wegman MAC [36]. That is, servers hold a secret-shared MAC tag for each broadcast channel, and only allow clients who know the valid MAC tag to write messages to the corresponding channel via DPF.

Servan-Schreiber *et al.* [32, 33] abstract the notion of PACLs for FSS from concrete applications. They improve Spectrum’s technique to more efficient PACLs for DPF, and propose a general PACL for P/poly FSS. However, their general PACL relies on costly secret-shared non-interactive proofs and is not quit efficient in practice.

1.3 Organization

Preliminaries can be found in Sec. 2. Our formal definitions of CDPF are presented in Sec. 3. The VDPF-PACL scheme in [33] is revisited in Sec. 4. A fast CDPF with logarithmic storage is presented in Sec. 5. Our wildcard-style CDPF is presented in Sec. 6. Our benchmark is shown in Sec. 7. Our impossibility and feasibility results of non-interactive CDPF are presented in Sec. 8.

In addition, we recap the definition of VDPF [12] in Appendix A. We recap the PACL definition in [33] and discuss its several problems in Appendix B. We revisit the VDPF-PACL scheme in [33], and present our attack and fix in Appendix 4. We present a non-interactive CDPF in the preprocessing model in Appendix D. Finally, we give some proof omitted from the main paper in Appendix E.

2 Preliminaries

Notations. Let λ denote the security parameter. Let p be a prime number. For $x \in \mathbb{Z}_p$, let $\llbracket x \rrbracket$ denote the additively secret sharing of the value x . Here, $\llbracket x \rrbracket \leftarrow \text{AddShare}_{\mathbb{Z}_p, n}(x)$ stands for generating n additive shares of x , where $\llbracket x \rrbracket := \{x^{(0)}, \dots, x^{(n-1)}\}$, $x^{(i)} \in \mathbb{Z}_p$ for all $i \in \mathbb{Z}_n$, and $x = \sum_{i \in \mathbb{Z}_n} x^{(i)}$ over $\mathbb{Z}_p$. $(\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket z \rrbracket) \leftarrow \text{Beaver}_{\mathbb{Z}_p, n}$ stands for generating a beaver triple, i.e., generating n additive shares of $x, y, z \leftarrow \mathbb{Z}_p$, such that $x \cdot y = z$. Let $\mathbb{G}$ be a circle group. For $x \in \mathbb{G}$, let $\langle x \rangle$ denote the multiplicatively secret sharing of the value x . Here, $\langle x \rangle \leftarrow \text{MulShare}_{\mathbb{G}, n}(x)$ stands for generating n multiplicative shares of x , where $\langle x \rangle := \{x^{(0)}, \dots, x^{(n-1)}\}$ and $x = \prod_{i \in \mathbb{Z}_n} x^{(i)}$ over $\mathbb{G}$.

Access Structure [2]. Let $\mathcal{P} := \{P_1, P_2, \dots, P_n\}$ be a set of parties. A collection $\mathbb{A} \subseteq 2^{\mathcal{P}}$ is monotone if $\forall B, C$: if $B \in \mathbb{A}$ and $B \subseteq C$, then $C \in \mathbb{A}$. An access structure $\mathbb{A}$ (respectively, monotone access structure) is a collection (respectively, monotone collection) of non-empty subsets of $\mathcal{P}$, i.e., $\mathbb{A} \subseteq 2^{\mathcal{P}} \setminus \{\emptyset\}$. We call the sets in $\mathbb{A}$, the *authorized sets*, and the ones not in $\mathbb{A}$, the *unauthorized sets*.

Function Secret Sharing. Function Secret Sharing, introduced by Boyle *et al.* [6], provides a way for a dealer to additively secret-share a function among evaluators. An FSS scheme consists of (at least) two algorithms **Gen**, **Eval**. The dealer generates secret shares of a function $f : \mathcal{D} \rightarrow \mathbb{G}$ by **Gen**. Using the function shares, evaluators can locally execute **Eval** to produce additive shares of $f(x)$ for any $x \in \mathcal{D}$, without learning information of f . DPF [15] is a useful FSS instance for the point function family.

Recently, a new notion, called *verifiable* DPF (VDPF), has been proposed to ensure the integrity of DPF against malicious dealer [3, 12]. On top of the conventional DPF security guarantees, a VDPF enables the evaluators to check whether the dealer’s inputs are well-formed point function. Informally, a VDPF consists of three PPT algorithms (**Gen**, **Eval**, **Verify**). The share generation algorithm **Gen** produces the secret shares of the point function $f_{\alpha, \beta}$. The verifiable evaluation algorithm **Eval** evaluates the function share over a set of inputs X , and produces the additive shares of the function outputs and a token. The verification function **Verify** takes input as tokens, and outputs 1 iff the function outputs have at most one non-zero item, which means that the reconstructed function is a well-formed point function. The formal VDPF definition can be found in Appendix A.

In addition, based on the DPF approach, Boyle *et al.* [3] presented a new primitive of *incremental* DPF (IDPF). More specifically, one can view the DPF as a secret-shared representation of a weight-1 vector of dimension 2^n ; while the IDPF gives a way to represent a binary tree with 2^n leaves whose nodes have non-zero value at a single path. In other words, the IDPF is an FSS scheme for the class of all-prefix point

function, parameterized by an integer n and Abelian groups $\{\mathbb{G}_1, \dots, \mathbb{G}_n\}$, and defined by

$$f : \bigcup_{i \in [n]} \{0, 1\}^i \rightarrow \bigcup_{i \in [n]} \mathbb{G}_i, \text{ given by}$$

$$f(x_1, \dots, x_i) = \begin{cases} \beta_i & \text{if } (x_1, \dots, x_i) = (\alpha_1, \dots, \alpha_i) \\ 0 & \text{otherwise} \end{cases}$$

where $\alpha \in \{0, 1\}^n$ and $\beta_1 \in \mathbb{G}_1, \dots, \beta_n \in \mathbb{G}_n$. We refer to α and $\beta := \{\beta_1, \dots, \beta_n\}$ as the *special input* and *special prefix outputs* of the function f . For convenience, we denote the all-prefix point function f as $f_{\alpha, \beta}$, and refer to $f_{\alpha, \beta}(x_1, \dots, x_i)$ as a prefix evaluation. Recently, the *verifiable* IDPF (VIDPF) has been proposed [10, 11, 27]. Similar to VDPF, VIDPF allows the evaluators to verify whether a pair of VIDPF keys can reconstruct a well-formed all-prefix point function. However, all previous VIDPF schemes only provide the single prefix evaluation algorithm. For our purpose, in Sec. 5, we will design a *batch prefix evaluation* scheme for VIDPF, which builds upon the state-of-the-art VIDPF technique from [27].

Bilinear maps and the BDH assumption. Let $\mathbb{G}, \mathbb{G}_t$ be two multiplicative cyclic groups of prime order p , and g be a generator of $\mathbb{G}$. We say e is a bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_t$, if it has two properties:

- Bilinearity: $e(u^a, v^b) = e(u, v)^{ab}$, $\forall u, v \in \mathbb{G}, a, b \in \mathbb{Z}_p$.
- Non-degeneracy: $e(g, g) \neq 1$.

The decisional Bilinear Diffie-Hellman (BDH) assumption [4, 17] is that, given a bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_t$, for $a, b, c, z \leftarrow \mathbb{Z}_p$, no PPT adversary can distinguish the tuple $(g^a, g^b, g^c, e(g, g)^{abc})$ and $(g^a, g^b, g^c, e(g, g)^z)$ with more than a negligible advantage.

XOR-collision resistant hash functions. Following the definition in [12], we say a function family $\mathcal{H}$ is XOR-collision resistant if no PPT adversary given a randomly sampled $h \leftarrow \mathcal{H}$ finds four values x_0, x_1, x_2, x_3 such that $(x_0, x_1) \neq (x_2, x_3)$, $(x_0, x_1) \neq (x_3, x_2)$ and $h(x_0) \oplus h(x_1) = h(x_2) \oplus h(x_3) \neq 0$ with more than a negligible probability.

3 DPF with Constraints

In this section, we present a formal definition of *DPF with constraints* (CDPF), which can be viewed as a generalization of the *Private Access Control Lists (PACL)* for DPF proposed in [32, 33]. A bit more concretely, we take two steps to enforce constraints for DPF.

- *Defining the constraints.* Let $\mathcal{F}$ be a function family of point functions. Let $\{\mathcal{Q}_i\}_{i \in \mathbb{Z}_N}$ be N number of disjoint subsets of authorized functions, that is, for any $i \neq j$, it holds that $\mathcal{Q}_i \cap \mathcal{Q}_j = \emptyset$. Now, for each distinct authorized subset $\mathcal{Q}_i \subseteq \mathcal{F}$, a constraint configuration Λ_i can be defined. Then for function family $\mathcal{F}$, a constraint configuration list $\Lambda := \{\Lambda_i\}$ is defined.
- *Enabling the verification of the constraints.* Next, an efficiently computable predicate **Check** is introduced to verify if a constraint configuration is valid. Concretely, given a function f and a secret key sk , we have $\text{Check}(\Lambda, f, \text{sk}) = 1$ iff f belongs to an authorized subset, say $\mathcal{Q}_i$, and (Λ_i, sk) belongs to an efficiently decidable binary relation R . In particular, each constraint configuration Λ_i may be a set of multiple entries and the secret key sk corresponding to any of entries satisfies $R(\Lambda_i, \text{sk}) = 1$. Throughout, we denote the number of entries as ℓ .

In this paper, we focus on 2-party CDPF (i.e., DPF with constraints), where several roles, including (i) *two* evaluators (also called servers) along with (ii) a data owner, and (iii) a dealer (also called user), are involved; please also see Fig. 1. Very briefly, the two evaluators P_0, P_1 can securely attest **Check** = 1 over the secret-shared function from the dealer D . Furthermore, similar to the verifiable DPF [12], we consider the malicious security for a single corrupted party. That is, the dealer and evaluators are non-colluding. Formally, we define the 2-party CDPF as follows:

Definition 1 (2-party DPF with constraints). *Let $\mathcal{F}$ be a family of point functions $f : \mathcal{D} \rightarrow \mathbb{G}$ where $\mathbb{G}$ is an Abelian group³, and N be an integer in $\text{poly}(\lambda)$ such that $N \leq |\mathcal{D}|$. A 2-party constrained distributed*

³Note that the order of $\mathbb{G}$ is not important in DPF and can even be 2.

point function for function family $\mathcal{F}$, consists of five PPT algorithms (Constraint, SKGen, FGen, VEval, Verify) as follows:

- (Λ, msk) or $\perp \leftarrow \text{Constraint}(1^\lambda, \{(Q_i, A_i)\}_{i \in \mathbb{Z}_N})$ is the constraint generation algorithm that takes input as the security parameter 1^λ and the set of function sets $Q_i \subseteq \mathcal{F}$ with the access structure A_i . It outputs a tuple of a constraint configuration list $\Lambda := \{\Lambda_i\}_{i \in \mathbb{Z}_N}$ and a master secret key msk , or a symbol $\perp$ for error.
- $\text{sk} \leftarrow \text{SKGen}(1^\lambda, \text{id}, \text{msk})$ is the secret key generation algorithm⁴ that takes input as the security parameter 1^λ , an identity description id and a master secret key msk . It outputs a secret key sk .
- $((f^{(0)}, \pi^{(0)}), (f^{(1)}, \pi^{(1)})) \leftarrow \text{FGen}(\Lambda, f, \text{sk})$ is the share generation algorithm that takes input as a constraint configuration list Λ , function $f \in \mathcal{F}$ and a secret key sk . It outputs a pair of function shares with proof shares.
- $(\{y_x^{(b)}\}_{x \in X}, \tau^{(b)}) \leftarrow \text{VEval}(b, \Lambda, f^{(b)}, \pi^{(b)}, X)$ is the verifiable evaluation algorithm that takes input as an index $b \in \{0, 1\}$, a constraint configuration list Λ , a function share $f^{(b)}$, a proof share $\pi^{(b)}$, and a set of function inputs $X \subseteq \mathcal{D}$. It outputs a tuple of values. The first set of values $\{y_x^{(b)}\}$ are DPF outputs, and the second item is an audit token $\tau^{(b)}$.
- $1 \text{ or } 0 \leftarrow \text{Verify}(\tau^{(0)}, \tau^{(1)})$ is the verification algorithm that takes input as two tokens. It outputs 1 or 0.

The above algorithms implicitly take group descriptions as their input. For readability, we omit them in the syntax when they are clear in the context. We say a constrained distributed point function $\text{CDPF} = (\text{Constraint}, \text{SKGen}, \text{FGen}, \text{VEval}, \text{Verify})$ for function family $\mathcal{F}$ is secure if it satisfies three properties as follows:

- **Completeness.** For any $\{(Q_i, A_i)\}_{i \in \mathbb{Z}_N}$, $L \leftarrow \text{Constraint}(1^\lambda, \{(Q_i, A_i)\}_{i \in \mathbb{Z}_N})$ such that $L \neq \perp$ and parse $L := (\Lambda, \text{msk})$, any $i \in \mathbb{Z}_N$, any $f \in Q_i$, any $\text{sk} \leftarrow \text{SKGen}(1^\lambda, \text{id}, \text{msk})$ where id belongs to A_i , and any $X \subseteq \mathcal{D}$, we have

$$\Pr \left[\begin{array}{l} ((f^{(0)}, \pi^{(0)}), (f^{(1)}, \pi^{(1)})) \leftarrow \text{FGen}(\Lambda, f, \text{sk}); \\ (\{y_x^{(0)}\}_{x \in X}, \tau^{(0)}) \leftarrow \text{VEval}(0, \Lambda, f^{(0)}, \pi^{(0)}, X); \\ (\{y_x^{(1)}\}_{x \in X}, \tau^{(1)}) \leftarrow \text{VEval}(1, \Lambda, f^{(1)}, \pi^{(1)}, X); \\ (\forall x \in X, y_x^{(0)} + y_x^{(1)} = f(x)) \wedge \\ (\text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1) \end{array} \right] = 1$$

- **Privacy.** For a constraint configuration list Λ , a function $f \in \mathcal{F}$, a secret key sk and a set of function inputs $X \subseteq \mathcal{D}$, define the distribution of the view of P_b as

$$\text{View}_{\text{CDPF}}(b, \Lambda, f, \text{sk}, X) := \{(f^{(b)}, \pi^{(b)}, \tau^{(1-b)})\}_\lambda$$

where $((f^{(0)}, \pi^{(0)}), (f^{(1)}, \pi^{(1)})) \leftarrow \text{FGen}(\Lambda, f, \text{sk})$ and $(Y, \tau^{(1-b)}) \leftarrow \text{VEval}(1-b, \Lambda, f^{(1-b)}, \pi^{(1-b)}, X)$.

There exists a PPT simulator Sim such that for any Λ, msk generated from $\text{Constraint}(1^\lambda, \{(Q_i, A_i)\}_{i \in \mathbb{Z}_N})$ for any $\{(Q_i, A_i)\}_{i \in \mathbb{Z}_N}$, any function f from any Q_i , any secret key $\text{sk} \leftarrow \text{SKGen}(1^\lambda, \text{id}, \text{msk})$ where id belongs to A_i , any $X \subseteq \mathcal{D}$, and any $b \in \{0, 1\}$, the following two distributions are computationally indistinguishable:

$$\text{View}_{\text{CDPF}}(b, \Lambda, f, \text{sk}, X) \approx_c \text{Sim}(1^\lambda, b, \Lambda, X)$$

- **Soundness.** For any PPT adversary $\mathcal{A}$, it holds that

$$\Pr[\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

where the game $\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}(\lambda)$ is depicted in Fig. 2.

Remark 1. In this section, we define the security properties of CDPF for the normal case where the verification result of P_b depends on both $\tau^{(0)}$ and $\tau^{(1)}$. For completeness, we define different security properties for the special case where the verification result of P_b only depends on $\tau^{(b)}$, which can be found in Appendix C.

Remark 2. We note that the privacy definition of [33] is problematic and does not capture the concept of rushing adversaries. Concretely, in the privacy definition in [33], the view of evaluator does not include the function share. It results in a trivial simulator (cf. the proof of Theorem 3 in [33]) who simulates the token independent of function share. In Appendix B, we provide more details; in particular, we provide a counterexample to illustrate this subtle problem.

⁴The representation of id depends on the concrete system. For example, in PACL [32, 33] exact identities are considered.

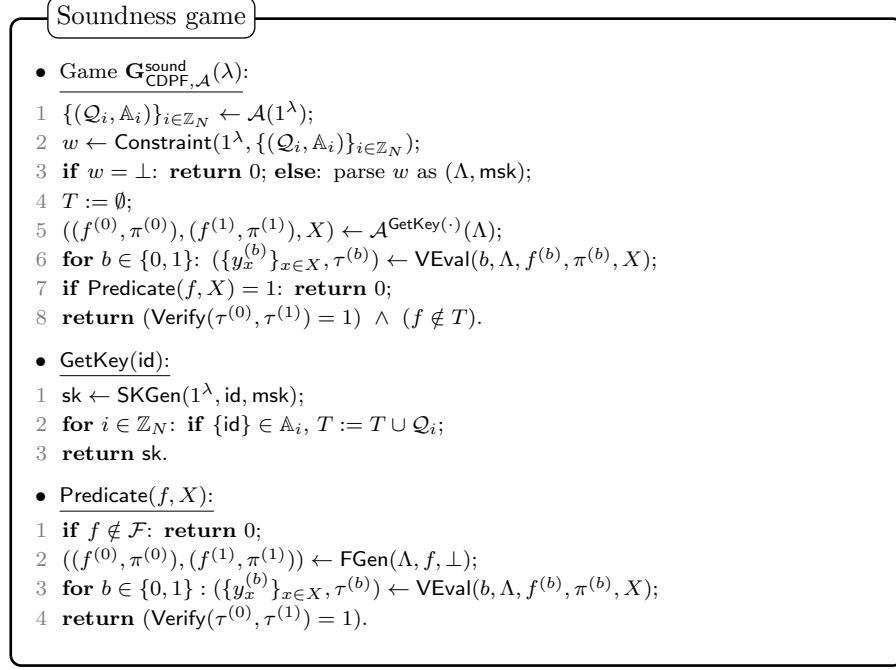


Figure 2: The soundness game for $\text{CDPF} = (\text{Constraint}, \text{SKGen}, \text{FGen}, \text{VEval}, \text{Verify})$ for function family $\mathcal{F}$.

4 Revisiting VDPF-PACL in [33]

The recent work [33] presents PACL schemes for the class of (verifiable) DPFs. In this section, we show their VDPF-PACL is not sound for the adversarially chosen β .

The VDPF-PACL scheme. Consider $\mathbb{G} := \mathbb{Z}_p^*$ as a group with order $p - 1$ and generator g in which the discrete logarithm problem is assumed to be computationally intractable.⁵ Without loss of generality, let $\ell = 1$ and focus on 2-party VDPF. For a family $\mathcal{F} := \{f : \{0, 1\}^n \rightarrow \mathbb{Z}_p\}$ of point functions and $N := 2^n$, the VDPF-PACL initiates constants $\Lambda := \{\text{vk}_i\}_{i \in \mathbb{Z}_N}$ by KeyGen algorithm:

- KeyGen($1^\lambda, f_{i, \beta} \in \mathcal{F}$):
 - 1 $\text{sk}_i \leftarrow \mathbb{Z}_{p-1}$, $\text{vk}_i := g^{\text{sk}_i}$;
 - 2 **return** $(\text{vk}_i, \text{sk}_i)$.

In [33], the VDPF-PACL uses *Schnorr Proof over Secret Shares* (SPoSS) as a building block. SPoSS is a discrete-logarithm zero-knowledge proof-of-knowledge over an additively secret-shared element. More specifically, it consists three PPT algorithms: (1) $\text{SPoSS.Prove}(x) \rightarrow (\pi^{(0)}, \pi^{(1)})$ takes the discrete logarithm x of w base g as input, and outputs the secret sharing of a non-interactive zero-knowledge proof; (2) $\text{SPoSS.Audit}(b, w^{(b)}, \pi^{(b)}) \rightarrow \tau^{(b)}$ takes the additive share of w and the SPoSS proof share as input, and outputs a verification token; (3) $\text{SPoSS.Verify}(\tau^{(0)}, \tau^{(1)})$ takes the tokens of two evaluators as input, and outputs 1 if and only if $w^{(0)} + w^{(1)} = g^x$ over the field $\mathbb{Z}_p$.

Here, we briefly describe the access process of VDPF-PACL based on SPoSS. First of all, the user chooses an point function $f_{\alpha, \beta} \in \mathcal{F}$, and secret-shares it to two servers using the VDPF technique. Meanwhile, the user plays the role of the prover to provide a SPoSS proof of the secret key $\text{sk} \in \mathbb{Z}_{p-1}$ for the secret-shared $\llbracket g^{\text{sk}} \rrbracket$ as follows:

- Prove($f_{\alpha, \beta}, \text{sk}$):
 - 1 $(\pi^{(0)}, \pi^{(1)}) \leftarrow \text{SPoSS.Prove}(\text{sk})$;

⁵Refer to Chapter 10 of [22] for how to pick a suitable p . For example, the largest prime dividing $p - 1$ is sufficiently large.

2 **return** $(\pi^{(0)}, \pi^{(1)})$.

Upon receiving the proof shares from the user, each server $P_b \in \{P_0, P_1\}$ as a verifier obviously selects the target verification key by the function share $f_{\alpha, \beta}^{(b)}$ and audits the SPoSS proof using **Audit** algorithm:

- **Audit** $(b, \Lambda, f_{\alpha, \beta}^{(b)}, \pi^{(b)})$:
 - 1 Parse $\Lambda := \{\text{vk}_i\}_{i \in \mathbb{Z}_N}$;
 - 2 $(\{y_i^{(b)}\}_{i \in \mathbb{Z}_N}, \tau_0^{(b)}) \leftarrow \text{VDPF.Eval}(b, f_{\alpha, \beta}^{(b)}, \mathbb{Z}_N)$;
 - 3 $w^{(b)} := \sum_{i=0}^{N-1} \text{vk}_i \cdot y_i^{(b)} \pmod{p}$;
 - 4 $\tau_1^{(b)} \leftarrow \text{SPoSS.Audit}(b, w^{(b)}, \pi^{(b)})$;
 - 5 **return** $\tau^{(b)} := (\tau_0^{(b)}, \tau_1^{(b)})$.

Finally, two evaluators exchange their audit tokens $\tau^{(0)}, \tau^{(1)}$, and verify the well-formedness of VDPF and the SPoSS proof using **Verify** algorithm:

- **Verify** $(\tau^{(0)}, \tau^{(1)})$:
 - 1 Parse $\tau^{(0)} := (\tau_0^{(0)}, \tau_1^{(0)})$, and $\tau^{(1)} := (\tau_0^{(1)}, \tau_1^{(1)})$;
 - 2 **return** $\text{VDPF.Verify}(\tau_0^{(0)}, \tau_0^{(1)})$
 $\wedge \text{SPoSS.Verify}(\tau_1^{(0)}, \tau_1^{(1)})$.

It is easy to see, **Verify** outputs 1 if and only if both the VDPF and the SPoSS verification pass.

The description of our attack. In light of the VDPF-PACL, the SPoSS just guarantees that the additively secret-shared $\llbracket w \rrbracket := g^{\text{sk}_\alpha} \cdot \llbracket \beta \rrbracket$ is equal to $g^{\llbracket \text{sk} \rrbracket}$ over the field $\mathbb{Z}_p$. This ignorance of β is vulnerable, and we show how to exploit it. We construct an adversary $\mathcal{A}$ who breaks the soundness of VDPF-PACL:

- $\mathcal{A}(1^\lambda, \Lambda)$:
 - 1 Parse $\Lambda := \{\text{vk}_i\}_{i \in \mathbb{Z}_N}$;
 - 2 $\alpha \leftarrow \{0, 1\}^n$, $r \leftarrow \mathbb{Z}_{p-1}$, $\beta := g^r \cdot \text{vk}_\alpha^{-1} \pmod{p}$;
 - 3 $(f_{\alpha, \beta}^{(0)}, f_{\alpha, \beta}^{(1)}) \leftarrow \text{VDPF.Gen}(1^\lambda, f_{\alpha, \beta})$;
 - 4 $(\pi^{(0)}, \pi^{(1)}) \leftarrow \text{Prove}(\Lambda, f_{\alpha, \beta}, r)$;
 - 5 **return** $((f_{\alpha, \beta}^{(0)}, f_{\alpha, \beta}^{(1)}), (\pi^{(0)}, \pi^{(1)}))$.

Using the VDPF keys $(f_{\alpha, \beta}^{(0)}, f_{\alpha, \beta}^{(1)})$, two evaluators obviously obtain a scaled verification key $\llbracket w \rrbracket := \Lambda_\alpha \cdot \llbracket \beta \rrbracket := \llbracket g^r \rrbracket$, and thus the proof π for x over w is verified by SPoSS. In addition, since $(f_{\alpha, \beta}^{(0)}, f_{\alpha, \beta}^{(1)})$ encodes a point function, the VDPF verification passes. Therefore, the algorithm **Verify** outputs 1 with the probability 1. In a word, $\mathcal{A}$ successfully forges a proof by choosing an appropriate $\beta \in \mathbb{Z}_p$.

Our fix. This soundness issue is based on the fact that the private value β of the point function is adversarially chosen from $\mathbb{Z}_p$, and it scales the verification key vk_α . Naively, if the evaluators additionally check $\beta = 1$, our attack is fixed. However, in many applications (e.g. the mailbox system [14]), one may need to use the customized β , such as anonymous communication. To overcome this problem, we introduce an auxiliary output. In particular, for any point function $f_{\alpha, \beta} : \{0, 1\}^n \rightarrow \mathbb{Z}_p$, we can construct a new point function $f_{\alpha, (\beta, 1)} : \{0, 1\}^n \rightarrow \mathbb{Z}_p^2$, where the tuple $(\beta, 1)$ denotes the new special output in the range $\mathbb{Z}_p^2$, and its second item 1 is named the special auxiliary output. We let the dealer secret-share a new point function $f_{\alpha, (\beta, 1)}$ via VDPF to be evaluated. After that, the evaluators use the auxiliary outputs to select the target verification key, and then jointly check if the special auxiliary output is equal to 1. Concretely, we adapted the **Audit** and **Verify** of the VDPF-PACL as follows:

- **Audit** $(b, \Lambda, f_{\alpha, (\beta, 1)}^{(b)}, \pi^{(b)})$:

```

1   $(\{y_i^{(b)}\}_{i \in \mathbb{Z}_N}, \tau_0^{(b)}) \leftarrow \text{VDPF.Eval}(b, f_{\alpha, (\beta, 1)}^{(b)}, \mathbb{Z}_N);$ 
2  for  $i \in \mathbb{Z}_N$ :  $\text{Parse } y_i^{(b)} := (y_{i,0}^{(b)}, y_{i,1}^{(b)}) \in \mathbb{Z}_p^2;$ 
3   $w^{(b)} := \sum_{i=0}^{N-1} \Lambda_i \cdot y_{i,1}^{(b)};$ 
4   $\tau_1^{(b)} \leftarrow \text{SPoSS.Audit}(b, w^{(b)}, \pi^{(b)});$ 
5   $\tau_2^{(b)} := H(b + (-1)^b \cdot \sum y_{i,1}^{(b)});$ 
6  return  $\tau^{(b)} := (\tau_0^{(b)}, \tau_1^{(b)}, \tau_2^{(b)}).$ 

```

• $\text{Verify}(\tau^{(0)}, \tau^{(1)})$:

```

1   $\text{Parse } \tau^{(0)} := (\tau_0^{(0)}, \tau_1^{(0)}, \tau_2^{(0)});$ 
2   $\text{Parse } \tau^{(1)} := (\tau_0^{(1)}, \tau_1^{(1)}, \tau_2^{(1)});$ 
3  return  $\text{VDPF.Verify}(\tau_0^{(0)}, \tau_0^{(1)})$ 
    $\wedge \text{SPoSS.Verify}(\tau_1^{(0)}, \tau_1^{(1)}) \wedge \tau_2^{(0)} = \tau_2^{(1)}.$ 

```

where $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a collision-resistant hash function. We note that, the equality of $\tau_2^{(0)} = \tau_2^{(1)}$ restricts the non-zero element in $\{y_{i,1}^{(0)} + y_{i,1}^{(1)}\}$ is equal to 1, which ensures $\llbracket w \rrbracket$ is always a valid (i.e., non-scaled) verification key in Λ . Therefore, the adversary cannot forge a proof π by scaling verification key.

5 CDPF with Logarithmic Storage

We observe that, the existing CDPF schemes (e.g. PACLs in [32, 33]) require that evaluators store at least N verification keys for N constraint configurations. This is because that their underlying DPF expands to a vector of length N to indicate the 1-out-of- N configuration selection. Furthermore, the selection step is the performance bottleneck when N is large. In this section, we show how to construct a CDPF scheme with logarithmic constraint storage size and logarithmic selection computation complexity.

For simplicity, we focus on the case where each constraint configuration has only one entry, i.e., $\ell = 1$, and our scheme generalizes to arbitrary ℓ . Intuitively, we use $2 \log N$ public keys to represent N verification keys as constraint configurations. Let $\mathbb{G}$ be a group with the generator g and $\alpha_1, \dots, \alpha_{\log N}$ be the bits of α . We set the public keys to $(k_{i,0} := g^{r_{i,0}}, k_{i,1} := g^{r_{i,1}})_{i \in [\log N]}$, and select the α -th verification key by $\text{vk}_\alpha := \prod_{i \in [\log N]} k_{i, \alpha_i}$. Only one who has the discrete-logarithm knowledge of vk_α can produce a valid proof for α -th verification key.

As we already discussed in the Sec. 1, there are two challenges to this intuition: (1) the evaluators are required to obliviously bit-decompose the special input α for configuration selection; (2) the secret key should not be the discrete logarithm of the corresponding verification key. To overcome these problems, we present a verifiable DPF scheme with layer outputs, called incremental VDPF, and introduce bilinear map to blind the discrete logarithm, respectively.

5.1 Incremental VDPF

For a point function $f_{\alpha, \beta}$ with input domain $\{0, 1\}^n$, consider a batch evaluation of $f_{\alpha, \beta}$ on any set of inputs $X \subseteq \{0, 1\}^n$. In a standard (verifiable) DPF scheme, given the function shares of $f_{\alpha, \beta}$, the evaluators obtain secret-shared function outputs (i.e., a vector of dimension $|X|$ that is non-zero at most a single point) after the batch evaluation. In an incremental VDPF scheme, the evaluators additionally obtain secret-shared layer outputs $\{z_{i,0}, z_{i,1}\}_{i \in [n]}$ such that for any $i \in [n], b \in \{0, 1\}$,

$$z_{i,b} = \begin{cases} \beta'_i & \text{if } \alpha_i = b \wedge \alpha_{[1,i]} \in X_{[1,i]} \\ 0 & \text{otherwise} \end{cases}$$

where α_i denotes the i -th bit of α , $\alpha_{[1,i]}$ denotes the i -bit prefix of α , and $X_{[1,i]}$ denotes the set of i -bit prefixes of all element in X . We refer to $\{\beta'_1, \dots, \beta'_n\}$ as the special layer outputs. We define the incremental VDPF as follows:

Definition 2 (Incremental VDPF). Let $\mathcal{F}$ be a function family of point functions $f : \{0, 1\}^n \rightarrow \mathbb{G}$ where $\mathbb{G}$ is an Abelian group, and $\{\mathbb{G}'_i\}_{i \in [n]}$ be Abelian groups. A 2-party incremental VDPF scheme, parameterized by $\mathcal{F}$ and $\{\mathbb{G}'_i\}_{i \in [n]}$, consists of three PPT algorithms (Gen, BEvalPref, BVerify):

- $(f_{\alpha, \beta}^{(0)}, f_{\alpha, \beta}^{(1)}) \leftarrow \text{Gen}(1^\lambda, f_{\alpha, \beta}, \{\beta'_i\}_{i \in [n]})$ is the share generation algorithm that takes input as a security parameter 1^λ , a point function $f_{\alpha, \beta} \in \mathcal{F}$ and the special layer outputs $\{\beta'_i \in \mathbb{G}'_i\}_{i \in [n]}$. It outputs a pair of IVDPF keys.
- $(\{y_x^{(b)}\}_{x \in X}, \{z_{i,0}, z_{i,1}\}_{i \in [n]}, \tau^{(b)}) \leftarrow \text{BEvalPref}(b, f_{\alpha, \beta}^{(b)}, X)$ is the verifiable evaluation algorithm that takes input as an index $b \in \{0, 1\}$, an IVDPF key $f_{\alpha, \beta}^{(b)}$ and a set of inputs $X \subseteq \{0, 1\}^n$. It outputs a tuple of values. The first set of values are additive shares of $f_{\alpha, \beta}(x), x \in X$. The second set of values are additive shares of layer outputs. The last item is a token.
- $1 \text{ or } 0 \leftarrow \text{BVerify}(\tau^{(0)}, \tau^{(1)})$ is the verification algorithm that takes input as two tokens. It outputs 1 or 0.

A secure IVDPF must satisfy three properties as follows:

Completeness. For any function $f_{\alpha, \beta} \in \mathcal{F}$, any special layer outputs $\{\beta'_i \in \mathbb{G}'_i\}_{i \in [n]}$, any $(f_{\alpha, \beta}^{(0)}, f_{\alpha, \beta}^{(1)}) \leftarrow \text{Gen}(1^\lambda, f_{\alpha, \beta}, \{\beta'_i\}_{i \in [n]})$ and any $X \subseteq \{0, 1\}^n$, it holds that

$$\Pr \left[\begin{array}{l} (Y^{(0)}, Z^{(0)}, \tau^{(0)}) \leftarrow \text{BEvalPref}(0, f_{\alpha, \beta}^{(0)}, X); \\ (Y^{(1)}, Z^{(1)}, \tau^{(1)}) \leftarrow \text{BEvalPref}(1, f_{\alpha, \beta}^{(1)}, X); \\ \forall x \in X, y_x^{(0)} + y_x^{(1)} = f(x) \wedge \\ \forall i \in [n], z_{i, \alpha_i}^{(0)} + z_{i, \alpha_i}^{(1)} = 0 \wedge \\ z_{i, \alpha_i}^{(0)} + z_{i, \alpha_i}^{(1)} = (\alpha_{[1, i]} \in X_{[1, i]}) \cdot \beta'_i \wedge \\ \text{BVerify}(\tau^{(0)}, \tau^{(1)}) = 1 \end{array} \right] = 1$$

where $Y^{(b)} := \{y_x^{(b)}\}_{x \in X}$, $Z^{(b)} := \{z_{i,0}, z_{i,1}\}_{i \in [n]}$ for $b \in \{0, 1\}$, α_i is the i -th bit of α , $\alpha_{[1, i]}$ is the i -bit prefix of α , and $X_{[1, i]}$ is the set of i -bit prefixes of all element in X .

Privacy. For a point function $f \in \mathcal{F}$, the special layer outputs $B := \{\beta'_i \in \mathbb{G}'_i\}_{i \in [n]}$, and a set of inputs $X \subseteq \{0, 1\}^n$, define the view of evaluator P_b $\text{View}_{\text{IVDPF}}(b, f, B, X)$ as a probability distribution ensemble $\{(f^{(b)}, \tau^{(1-b)})\}_\lambda$, where the function shares $(f^{(0)}, f^{(1)})$ are generated from $\text{Gen}(1^\lambda, f, B)$, and the audit token $\tau^{(1-b)}$ is computed by $\text{BEvalPref}(1-b, f^{(1-b)}, X)$. There exists a PPT simulator Sim such that for all $f \in \mathcal{F}, B := \{\beta'_i \in \mathbb{G}'_i\}_{i \in [n]}, X \subseteq \{0, 1\}^n$, the two distributions are computationally indistinguishable:

$$\text{View}_{\text{IVDPF}}(b, f, B, X) \approx_c \text{Sim}(1^\lambda, b, n, \mathbb{G}, \{\mathbb{G}'_i\}_{i \in [n]}, X)$$

Soundness. Let $f^{(b)}$ be the (possibly maliciously generated) function-share received by the evaluator P_b . There exists a negligible function negl such that for any $X \subseteq \{0, 1\}^n$ and $(\{y_x^{(b)}\}_{x \in X}, \{z_{i,0}, z_{i,1}\}_{i \in [n]}, \tau^{(b)}) \leftarrow \text{BEvalPref}(b, f^{(b)}, X)$ for $b \in \{0, 1\}$, it holds that

$$\Pr \left[\begin{array}{l} \text{BVerify}(\tau^{(0)}, \tau^{(1)}) = 1 : \\ \left| \{x \in X \mid y_x^{(0)} + y_x^{(1)} \neq 0\} \right| \leq 1 \wedge \\ \forall i \in [n], |C_i| \geq 1 \wedge \\ \left(\begin{array}{l} \forall x \in X, y_x^{(0)} + y_x^{(1)} = 0 \vee \\ \forall i \in [n], x_i \in C_i \end{array} \right) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

where $C_i := \{0, 1\} \setminus \{1 - j \mid z_{j,0}^{(0)} + z_{j,0}^{(1)} \neq 0, j \in \{0, 1\}\}$ and x_i stands for the i -th bit of x .

Our IVDPF construction is inspired by two techniques: the first is called “constant PRG optimization for distributed DPF generation” in [13], and the second is “incremental DPF” in [3]. We below will provide a high-level explanation of our construction ideas. Recap the DPF representation in [7]. For the point function $f_{\alpha, \beta}$, each of the two DPF keys defines a GGM-style binary tree [16] with 2^n leaves, and every node in the tree is labeled by a pseudo-random value. The DPF construction guarantees that, if a node is in the path

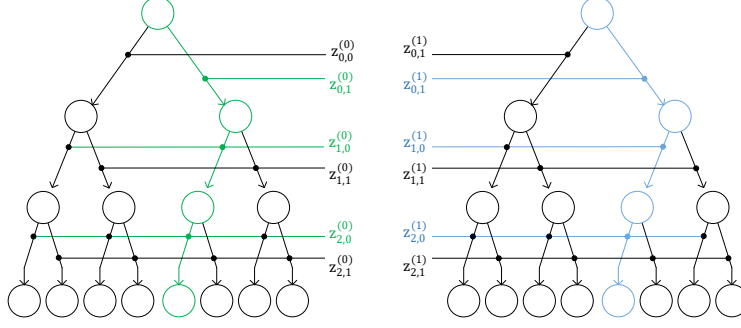


Figure 3: DPF with layer outputs

from root to the α -th leaf, its labels in two trees are independent; otherwise, its labels are identical. Let α_i be the i -th bit of α and $\bar{\alpha}_i := 1 - \alpha_i$. As shown in Fig. 3, for the sums $z_{i,0}^{(b)}, z_{i,1}^{(b)}$ of all left and right children at level i , we have $z_{i,\alpha_i}^{(0)} \neq z_{i,\alpha_i}^{(1)}$ and $z_{i,\bar{\alpha}_i}^{(0)} = z_{i,\bar{\alpha}_i}^{(1)}$. Therefore, we can instruct the i -th layer output by making $z_{i,\alpha_i}^{(0)}, z_{i,\alpha_i}^{(1)}$ reconstruct β'_i and correcting $z_{i,\bar{\alpha}_i}^{(0)}, z_{i,\bar{\alpha}_i}^{(1)}$ to 0.

More specifically, our IVDPF construction is depicted in Fig. 4. In particular, the label of each node includes a pseudo-random seed $\tilde{s}^{(b)}$ and a control bit $t^{(b)}$. They generate the labels of children with the help of the corresponding correction word cw . As observed in [3], using the seed $\tilde{s}^{(b)}$ directly in the generation of both children and layer output would compromise its pseudo-randomness, which is required for security. Therefore, an extra PRG evaluation is employed to expand $\tilde{s}^{(b)}$ to a new pseudo-random seed $s^{(b)}$ for children and an element $w^{(b)}$ for the layer output.

Defending against a malicious dealer. The VDPF technique in [12] can ensure the dealer shares a well-formed point function. However, a malicious dealer may also attempt to cause a mismatch between the layer outputs and the point function outputs. For instance, the dealer secret-shares a point function $f_{\alpha,\beta}$ to two evaluators, while the layer outputs embedded in the function shares correspond to the binary representation of a different integer $\alpha' \neq \alpha$. To defend against this attack, we enable evaluators to verify that two trees defined by a pair of IVDPF keys only differ at one node per level. It forces that the nodes with different labels exactly form the special path. More specifically, we follow the verification method of the standard VDPF scheme in [12] to check each level. During the key generation, for each level i , we introduce a verification correction seed cs_i to correct the hash values of the different labels (i.e., the seeds and control bits) to be identical. In the verifiable evaluation, we let the evaluators check that all the corrected hash values are equal. If a correction seed can only correct at most one difference, this equality check guarantees that all other nodes at level i have the same labels on two trees.

Security. We show the security of our IVDPF in Fig. 4 with the following theorem, and its proof can be found in Appendix E.1.

Theorem 1. *Let $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda+2}$, $G'_0 : \{0, 1\}^\lambda \rightarrow \mathbb{G}$, and $G'_i : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda \times \mathbb{G}'_i$ for $i \in [n]$ be pseudorandom generators. Let $H : \{0, 1\}^{\lambda+1+n} \rightarrow \{0, 1\}^{4\lambda}$ be a collision resistant and XOR-collision resistant hash function, and $H' : \{0, 1\}^{4\lambda} \rightarrow \{0, 1\}^{2\lambda}$ be a collision-resistant hash function. The IVDPF scheme in Fig. 4 is a secure IVDPF as described in Definition 2.*

5.2 Our construction using incremental VDPF

We describe our CDPF scheme from incremental VDPF (IVDPF-CDPF) in Fig. 5. It is aimed to constraint a function family $\mathcal{F} := \{f : \{0, 1\}^n \rightarrow \tilde{\mathbb{G}}\}$ of point functions, where $\tilde{\mathbb{G}}$ is an Abelian group. Let $N := 2^n$. Our IVDPF-CDPF supports N constraint configurations while evaluators store only $2n$ public keys $\{k_{i,0}, k_{i,1}\}_{i \in [n]}$. The intuition is that, for a function $f_{\alpha,\beta}$ shared in the IVDPF scheme, the evaluators jointly select the verification key $\text{vk}_\alpha := \prod_{i \in [n]} (k_{i,0})^{z_{i,0}} \cdot (k_{i,1})^{z_{i,1}}$ using the layer outputs, and check if the dealer holds a secret key encoding the discrete logarithm of vk_α .

To defend against the attack on the linear composition of discrete logarithms, we utilize a bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_t$. Let g be the generator of $\mathbb{G}$. We let **Constraint** generate random public keys

Construction IVDPF

- **Parameters:** Let $\mathbb{G}, \mathbb{G}'_1, \dots, \mathbb{G}'_n$ be Abelian groups. Let $\mathcal{F} := \{f : \{0, 1\}^n \rightarrow \mathbb{G}\}$ be a function family of point functions. Let $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda+2}$, $G'_0 : \{0, 1\}^\lambda \rightarrow \mathbb{G}$, and $G'_i : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda \times \mathbb{G}'_i$ for $i \in [n]$ be pseudorandom generators. Let $H : \{0, 1\}^{\lambda+1+n} \rightarrow \{0, 1\}^{4\lambda}$ be a hash function sampled from a family $\mathcal{H}$ that is collision-resistant and XOR-collision-resistant, and $H' : \{0, 1\}^{4\lambda} \rightarrow \{0, 1\}^{2\lambda}$ be a hash function sampled from a family $\mathcal{H}'$ that is collision-resistant.
- **Gen**($1^\lambda, f_{\alpha, \beta} \in \mathcal{F}, \{\beta'_i \in \mathbb{G}'_i\}_{i \in [n]}$)
 - 1 Let $\alpha_1, \dots, \alpha_n$ be the bits of α ;
 - 2 Sample random $s_0^{(0)}, s_0^{(1)} \leftarrow \{0, 1\}^\lambda$;
 - 3 $t_0^{(0)} := 0; t_0^{(1)} := 1$;
 - 4 **for** $i := 1$ to n :
 - 5 $s_L^{(b)} || s_R^{(b)} || t_L^{(b)} || t_R^{(b)} \leftarrow G(s_{i-1}^{(b)})$ for $b \in \{0, 1\}$;
 - 6 **if** $\alpha_i = 0$: **keep** $:= L$; **lose** $:= R$;
 - 7 **else**: **keep** $:= R$; **lose** $:= L$;
 - 8 $t_L^{\text{cw}} := t_L^{(0)} \oplus t_L^{(1)} \oplus \alpha_i \oplus 1$;
 - 9 $t_R^{\text{cw}} := t_R^{(0)} \oplus t_R^{(1)} \oplus \alpha_i$;
 - 10 $s^{\text{cw}} := s_{\text{lose}}^{(0)} \oplus s_{\text{lose}}^{(1)}$; $\text{cw}_i := (s^{\text{cw}}, t_L^{\text{cw}}, t_R^{\text{cw}})$;
 - 11 $\tilde{s}_i^{(b)} := s_{\text{keep}}^{(b)} \oplus t_{i-1}^{(b)} \cdot s^{\text{cw}}$ for $b \in \{0, 1\}$;
 - 12 $t_i^{(b)} := t_{\text{keep}}^{(b)} \oplus t_{i-1}^{(b)} \cdot t_{\text{keep}}^{\text{cw}}$ for $b \in \{0, 1\}$;
 - 13 $\alpha_{[1, i]} := \alpha_1 || \dots || \alpha_i$;
 - 14 $\text{cs}_i := H(\tilde{s}_i^{(0)} || t_i^{(0)} || \alpha_{[1, i]}) \oplus H(\tilde{s}_i^{(1)} || t_i^{(1)} || \alpha_{[1, i]})$;
 - 15 $s_i^{(b)} || w_i^{(b)} \leftarrow G'_i(\tilde{s}_i^{(b)})$;
 - 16 $\text{lcw}_i := (-1)^{t_i^{(1)}} \cdot [\beta'_i - w_i^{(0)} + w_i^{(1)}]$;
 - 17 $\text{ocw} := (-1)^{t_n^{(1)}} \cdot (\beta - G'_0(s_n^{(0)}) + G'_0(s_n^{(1)}))$;
 - 18 $f_{\alpha, \beta}^{(0)} := (s_0^{(0)}, (\text{cw}_i, \text{cs}_i, \text{lcw}_i)_{i \in [n]}, \text{ocw})$;
 - 19 $f_{\alpha, \beta}^{(1)} := (s_0^{(1)}, (\text{cw}_i, \text{cs}_i, \text{lcw}_i)_{i \in [n]}, \text{ocw})$;
 - 20 **return** $(f_{\alpha, \beta}^{(0)}, f_{\alpha, \beta}^{(1)})$.
- **Eval**($b, f_{\alpha, \beta}^{(b)}, X \subseteq \{0, 1\}^n$):
 - 1 Parse $f_{\alpha, \beta}^{(b)} := (s_0, t_0, (\text{cw}_i, \text{lcw}_i)_{i \in [n]}, \text{ocw})$;
 - 2 $t_0 := b$; $\tau_i := \text{cs}_i$; $\tau^{(b)} := 0$;
 - 3 **for** $i \in [n]$: $z_{i,0}^{(b)} := 0$; $z_{i,1}^{(b)} := 0$.
 - 4 **for** $x \in X$:
 - 5 Let $x_1, \dots, x_n$ be the bits of x ;
 - 6 **for** $i := 1$ to n :
 - 7 $s_L || s_R || t_L || t_R \leftarrow G(s_{i-1})$;
 - 8 Parse $\text{cw}_i := (s^{\text{cw}}, t_L^{\text{cw}}, t_R^{\text{cw}})$;
 - 9 **if** $x_i = 0$: **keep** $:= L$;
 - 10 **else**: **keep** $:= R$;
 - 11 $\tilde{s}_i || t_i := (s_{\text{keep}} || t_{\text{keep}}) \oplus t_{i-1} \cdot (s^{\text{cw}} || t_{\text{keep}}^{\text{cw}})$;
 - 12 $x_{[1, i]} := x_1 || \dots || x_i$;
 - 13 $\tau_i := \tau_i \oplus H'(\tau_i \oplus H(\tilde{s}_i || t_i || x_{[1, i]})) \oplus t_i \cdot \text{cs}_i$;
 - 14 $s_i || w_i \leftarrow G'_i(\tilde{s}_i)$;
 - 15 $z_{i, x_i}^{(b)} := z_{i, x_i}^{(b)} + (-1)^b \cdot (w_i + t_i \cdot \text{lcw}_i)$;
 - 16 $y_x^{(b)} := (-1)^b \cdot (G'_0(s_n) + t_n \cdot \text{ocw})$;
 - 17 **for** $i \in [n]$: $\tau^{(b)} := \tau^{(b)} \oplus H'(\tau^{(b)} \oplus \tau_i)$;
 - 18 **return** $(\{y_x^{(b)}\}_{x \in X}, \{z_{i,0}^{(b)}, z_{i,1}^{(b)}\}_{i \in [n]}, \tau^{(b)})$.
 - **Verify**($\tau^{(0)}, \tau^{(1)}$)
 - 1 **return** $\tau^{(0)} = \tau^{(1)}$.

Figure 4: The construction of incremental VIDPF.

($k_{i,0} := e(g, g)^{r_{i,0}}, k_{i,1} := e(g, g)^{r_{i,1}}\rangle_{i \in [n]}$ to represent configurations, and choose a random group element $h := e(g, g)^s$. Next, SKGen can product randomized secret keys using the master secret key $\text{msk} := (\langle r_{i,0}, r_{i,1} \rangle_{i \in [n]}, s)$. For each vk_α , we have $d := \sum_{i \in [n]} r_{i, \alpha_i}$ such that $e(g, g)^d = \text{vk}_\alpha$. A random secret key for vk_α is computed by $\text{sk}_\alpha := (g^c, g^{d/c}, g^{s/c})$ where $c \leftarrow \mathbb{Z}_p^*$. Note that, we have $e(g^c, g^{d/c}) = e(g, g)^d$ and $e(g^c, g^{s/c}) = e(g, g)^s = h$.

To secretly share a point function $f_{\alpha, \beta}$, the dealer uses algorithm FGen to generate function shares and proof shares. Concretely, the dealer first constructs an all-prefix point function $f_{\alpha, 1 || \beta}$ with special input α and prefix special outputs $(\beta'_i)_{i \in [n]}$ where $\beta'_i := 1$ for $i \in [n-1]$ and $\beta'_n := (1, \beta)$. The dealer then computes VIDFP keys of $f_{\alpha, 1 || \beta}$ as function shares. Finally, it re-randomizes $\text{sk}_\alpha := (g^c, g^{d/c}, g^{s/c})$ to (u, v, w) such that $e(u, v) = e(g^c, g^{d/c})$ and $e(u, w) = e(g^c, g^{s/c})$, and produces the proof shares $\pi^{(0)}, \pi^{(1)}$, each consisting of u and **additive shares** of group elements v, w .

Upon receiving the function shares and the proof shares, the evaluators run the algorithm VEval over the constraint configurations (i.e., $2n$ public keys) and a set of function inputs $X \subseteq \{0, 1\}^n$. Then they obtain the secret shares of the function outputs $f_{\alpha, \beta}(x)$ for $x \in X$, as well as the tokens for constraint verification. More specifically, the secret-shared function outputs are parsed from the n -bit prefix evaluation results $\{y_{n,x}\}_{x \in X}$ of the VIDPF keys of $f_{\alpha, 1 || \beta}$. For the tokens, the evaluators first compute the bit decomposition of α from VIDPF for selecting $\langle \text{vk}_\alpha \rangle$, then jointly compute $\langle e(u, g^v) / \text{vk}_\alpha \rangle, \langle u^w \rangle$ and interpret these secret shares as tokens $\tau_2^{(b)}, \tau_3^{(b)}$. In addition, they computes the hash of u and set it as the last token $\tau_4^{(b)}$. After that, they can exchange tokens to confirm the consistency of u , and check if $e(u, v) = \text{vk}_\alpha$ and $e(u, w) = h$. Based on these checks, they make the verification decision. Note that, $e(u, w) = h$ restricts that u is (computed) from

Construction IVDPF-CDPF

- **Parameters:** Let $\tilde{\mathbb{G}}$ be an Abelian group, $\mathcal{F} := \{f : \{0, 1\}^n \rightarrow \tilde{\mathbb{G}}\}$ be a point function family, s.t. $N := 2^n$ is an integer in $\text{poly}(\lambda)$. Let $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_t$ be a bilinear map, where $\mathbb{G}, \mathbb{G}_t$ are multiplicative cyclic groups with the prime order p , and the generator g of $\mathbb{G}$. Let $H : \mathbb{G}_t \rightarrow \{0, 1\}^{2\lambda}$, $H' : \{0, 1\}^{n \cdot \lceil \log p \rceil} \rightarrow \{0, 1\}^{2\lambda}$ and $\tilde{H} : \mathbb{G} \rightarrow \{0, 1\}^{2\lambda}$ be hash functions sampled from $\mathcal{H}, \mathcal{H}', \tilde{\mathcal{H}}$ that are collision-resistant.
- **Constraint**($1^\lambda, \{\mathcal{Q}_i, \mathbb{A}_i\}_{i \in \mathbb{Z}_N}$):


```

1  assert  $\mathcal{Q}_i = \{f_{i,v}\}_{v \in \tilde{\mathbb{G}}}$ ;
2  for  $j \in [n]$ :
    /*generating master secret key*/
4     $r_{j,0}, r_{j,1} \leftarrow \mathbb{Z}_p$ ;
5     $k_{j,0} := e(g, g)^{r_{j,0}}, k_{j,1} := e(g, g)^{r_{j,1}}$ ;
6     $s \leftarrow \mathbb{Z}_p$ ;  $h := e(g, g)^s$ ;
7     $\text{msk} := (\{r_{j,0}, r_{j,1}\}_{j \in [n]}, \{\mathbb{A}_i\}_{i \in \mathbb{Z}_N}, s)$ ;
8    return  $(\Lambda := (\{k_{j,0}, k_{j,1}\}_{j \in [n]}, h), \text{msk})$ .
```
- **SKGen**($1^\lambda, \text{id}, \text{msk}$):


```

1  Parse  $\text{msk} := (\{r_{j,0}, r_{j,1}\}_{j \in [n]}, \{\mathbb{A}_i\}_{i \in \mathbb{Z}_N}, s)$ 
2   $\text{sk} := \emptyset$ ;
3  for  $i \in \mathbb{Z}_N$ :
4    Let  $i_1, \dots, i_n$  be the bits of  $i$ ;
5    if  $\{\text{id}\} \in \mathbb{A}_i$ :
      /*computing the dlog of  $\text{vk}_i$ */
6       $d := \sum_{j \in [n]} r_{j,i_j}$ ;
7       $c \leftarrow \mathbb{Z}_p^*$ ;  $\text{sk}_{i,0} := g^c$ ;
8       $\text{sk}_{i,1} := g^{d/c}$ ;  $\text{sk}_{i,2} := g^{s/c}$ ;
9       $\text{sk}_i := (\text{sk}_{i,0}, \text{sk}_{i,1}, \text{sk}_{i,2})$ ;
10      $\text{sk} := \text{sk} \cup \{\text{sk}_i\}$ ;
11  return  $\text{sk}$ .
```
- **FGen**($1^\lambda, \Lambda, f_{\alpha,\beta} \in \mathcal{F}, \text{sk}$):


```

1  Generate a list  $(\beta'_i)_{i \in [n]}$  s.t.  $\beta'_i := 1$  for  $i \in [n-1]$ 
   and  $\beta'_n := (1, \beta)$ 
2   $(f_{\alpha,1||\beta}, f_{\alpha,1||\beta}^{(1)}) \leftarrow \text{IVDPF.Gen}(1^\lambda, \alpha, (\beta'_i)_{i \in [n]})$ ;
3  Choose  $\text{sk}_\alpha$  from  $\text{sk}$ , and parse  $\text{sk}_\alpha := (u, v, w)$ ;
4   $t \leftarrow \mathbb{Z}_p$ ;
5   $u := u^t$ ;  $v := v^{1/t}$ ;  $w := w^{1/t}$ ;
6   $v^{(0)} \leftarrow \mathbb{G}$ ;  $v^{(1)} := v/v^{(0)}$ ;
7   $w^{(0)} \leftarrow \mathbb{G}$ ;  $w^{(1)} := w/w^{(0)}$ ;
8  for  $b \in \{0, 1\}$ :  $\pi^{(b)} := (u, v^{(b)}, w^{(b)})$ ;
9  return  $((f_{\alpha,1||\beta}^{(0)}, \pi^{(0)}), (f_{\alpha,1||\beta}^{(1)}, \pi^{(1)}))$ .
```
- **VEval**($b, \Lambda, f_{\alpha,1||\beta}^{(b)}, \pi^{(b)}, X \subseteq \{0, 1\}^n$):


```

1  Parse  $\pi^{(b)} := (u, v^{(b)}, w^{(b)})$ ;
2  Parse  $\Lambda := (\{k_{i,0}, k_{i,1}\}_{i \in [n]}, h)$ ;
3  Let  $X_{[1,i]}$  be the set of  $i$ -bit prefixes of the elements
   in  $X$ ;
4   $(\{y_{i,x}^{(b)}\}_{i \in [n], x \in X_{[1,i]}}, \tau_0^{(b)}) \leftarrow$ 
   IVDPF.BEvalPref( $b, f_{\alpha,1||\beta}^{(b)}, X$ );
   /* bit decomposition from VIDPF */
5  for  $i \in [n], x \in X_{[1,i]}$ :
6    if  $i = n$ : Parse  $y_{n,x}^{(b)} := (q_{n,x}^{(b)}, y_x^{(b)})$ ;
7    else: Set  $q_{i,x}^{(b)} := y_{i,x}^{(b)}$ ;
8     $z_{i,x}^{(b)} := z_{i,x}^{(b)} + q_{i,x}^{(b)}$ ;
9  Set  $\tau_1^{(b)} := 0$ ;
10 for  $i \in [n]$ :
11    $\delta_i^{(b)} := b + (-1)^b \cdot (z_{i,0}^{(b)} + z_{i,1}^{(b)})$ ;
12    $\tau_1^{(b)} := H'(\delta_1^{(b)} || \dots || \delta_n^{(b)})$ ;
   /*selecting the target  $\text{vk}_\alpha$ */
13    $\text{vk}^{(b)} := \prod_{i \in [n]} (k_{i,0})^{z_{i,0}^{(b)}} \cdot (k_{i,1})^{z_{i,1}^{(b)}}$ ;
14    $v := v^{(b)}$ ;  $w := w^{(b)}$ ;
15   if  $b = 0$ :
16      $\tau_2^{(b)} := H(e(u, v)/\text{vk}^{(b)})$ ;  $\tau_3^{(b)} := H(e(u, w))$ ;
17   else:  $\tau_2^{(b)} := H(\text{vk}^{(b)}/e(u, v))$ ;  $\tau_3^{(b)} := H(h/e(u, w))$ ;
18    $\tau^{(b)} := (\tau_0^{(b)}, \tau_1^{(b)}, \tau_2^{(b)}, \tau_3^{(b)}, \tau_4^{(b)} := \tilde{H}(u))$ ;
19   return  $(\{y_x^{(b)}\}_{x \in X}, \tau^{(b)})$ .
```
- **Verify**($\tau^{(0)}, \tau^{(1)}$):


```

1  return IVDPF.BVerify( $\tau_0^{(0)}, \tau_0^{(1)}$ )  $\wedge$ 
    $\forall i \in [4], \tau_i^{(0)} = \tau_i^{(1)}$ .
```

Figure 5: The construction of faster CDPF from IVDPF.

a real secret key generated by **Constraint**.

To ensure the dealer knows a valid secret key if **Verify** outputs 1, the evaluators additionally check if they obtain a valid verification key vk_α . They first verify the well-formedness of function shares using VIDPF tokens $\tau_0^{(0)}, \tau_0^{(1)}$, which restricts that the non-zero values in batch prefix evaluation results over the input set X are consistent with a single special input. Regarding these prefix evaluation results as a binary tree, for each level $i \in [n]$, each evaluator P_b sums values of all left nodes and right nodes to obtain $z_{i,0}^{(b)}, z_{i,1}^{(b)}$, respectively. We note that, if the VIDPF is well-formed, there is at most one non-zero value in $\{z_{i,0}, z_{i,1}\}$, which represents the i -th bit of α and is used to select the verification key vk_α . Next, the evaluators check if the non-zero value in $\{z_{i,0}, z_{i,1}\}$ is equal to 1. More specifically, each evaluator P_b computes the subtractive share of the difference between the value 1 and the sum of left and right nodes as $\delta_i := b + (-1)^b \cdot (z_{i,0}^{(b)} + z_{i,1}^{(b)})$. Then P_b hashes these subtractive shares to obtain the audit token $\tau_1^{(b)}$. Clearly, $\tau_1^{(0)} = \tau_1^{(1)}$ holds iff $\forall i \in [n], z_{i,\alpha_i}^{(0)} + z_{i,\alpha_i}^{(1)} = 1$.

Construction $\star$ -CDPF

- **Parameters:** Let ℓ, t be integers in $\text{poly}(\lambda)$. Let $\mathcal{F} := \{f : \{0, 1\}^n \rightarrow \mathbb{F}_{2^t}\}$, $\mathcal{F}' := \{f : \{0, 1\}^{n+\log \ell} \rightarrow \mathbb{F}_{2^t}\}$ be point function families, s.t. $N := 2^n$ is an integer in $\text{poly}(\lambda)$. Let $H : \{0, 1\}^{Nt} \rightarrow \{0, 1\}^{2\lambda}$ be a hash function sampled from a family $\mathcal{H}$ that is collision-resistant.
- The SKGen algorithm always outputs a valid secret key $\perp$.
- **Constraint**($1^\lambda, \{Q_i, \mathbb{A}_i\}_{i \in \mathbb{Z}_N}$):
 - 1 **assert** Q_i is represented by $\{\mathbf{rs}_{i,j} \in \mathbb{F}_{2^t}\}_{j \in \mathbb{Z}_\ell}$
 - 2 $\Lambda_i := \{\mathbf{rs}_{i,j}\}_{j \in \mathbb{Z}_\ell}, \forall i \in \mathbb{Z}_N$;
 - 3 **return** ($\Lambda := \{\Lambda_i\}_{i \in \mathbb{Z}_N}, \text{msk} := \emptyset$).
- **FGen**($\Lambda, f_{\alpha,\beta} \in \mathcal{F}, \perp$):
 - 1 $(f_{\alpha,\beta}^{(0)}, f_{\alpha,\beta}^{(1)}) \leftarrow \text{VDPF.Gen}(1^\lambda, f_{\alpha,\beta} \in \mathcal{F})$;
 - 2 Select $\mathbf{rs}_{\alpha,\rho}$ from Λ , s.t., $\beta \wedge \mathbf{rs}_{\alpha,\rho} = 0$;
 - 3 $\eta := \alpha \cdot \ell + \rho$;
 - 4 $(f_{\eta,\beta}^{(0)}, f_{\eta,\beta}^{(1)}) \leftarrow \text{VDPF.Gen}(1^\lambda, f_{\eta,\beta} \in \mathcal{F}')$;
 - 5 $\pi^{(0)} := f_{\eta,\beta}^{(0)}; \pi^{(1)} := f_{\eta,\beta}^{(1)}$;
 - 6 **return** $((f_{\alpha,\beta}^{(0)}, \pi^{(0)}), (f_{\alpha,\beta}^{(1)}, \pi^{(1)}))$.
- **VEval**($b, \Lambda, f_{\alpha,\beta}^{(b)}, \pi^{(b)}, X \subseteq \mathbb{Z}_N$):
 - 1 $(\{y_i^{(b)}\}, \tau_0^{(b)}) \leftarrow \text{VDPF.Eval}(b, f_{\alpha,\beta}^{(b)}, \mathbb{Z}_N)$;
 - 2 $(\{\tilde{y}_i^{(b)}\}, \tau_1^{(b)}) \leftarrow \text{VDPF.Eval}(b, \pi^{(b)}, \mathbb{Z}_N)$;
 - 3 $\tau_2^{(b)} := 0$;
 - 4 **for** $i \in \mathbb{Z}_N$: $\delta_i := y_i^{(b)} \oplus (\bigoplus_{j \in \mathbb{Z}_\ell} \tilde{y}_{i \cdot \ell + j}^{(b)})$;
 - 5 $\tau_2^{(b)} := H(\delta_0 || \dots || \delta_{N-1})$;
 - 6 $\tau_3^{(b)} := H(\bigoplus_{i \in \mathbb{Z}_N, j \in \mathbb{Z}_\ell} \mathbf{rs}_{i,j} \wedge \tilde{y}_{i \cdot \ell + j}^{(b)})$;
 - 7 **return** $(\{y_x^{(b)}\}_{x \in X}, \tau^{(b)} := \{\tau_i^{(b)}\}_{i \in [0,3]})$.
- **Verify**($\tau^{(0)}, \tau^{(1)}$):
 - 1 Parse $\tau^{(b)} := \{\tau_i^{(b)}\}_{i \in [0,3]}$ for $b \in \{0, 1\}$;
 - 2 **return** $\forall i \in \{0, 1\}, \text{VDPF.Verify}(\tau_i^{(0)}, \tau_i^{(1)})$
 $\wedge \forall i \in \{2, 3\}, \tau_i^{(0)} = \tau_i^{(1)}$.

Figure 6: The construction of $\star$ -CDPF.

Note that, in the algorithm **VEval**, the function shares are evaluated on the input set $X \subseteq \mathbb{Z}_N$ rather than the full domain. A malicious dealer might try to bypass the verification by letting $\alpha \notin X$. However, the equality check of $\tau_1^{(0)} = \tau_1^{(1)}$ restricts the special input $\alpha \in X$. Concretely, if $\alpha \notin X$, then there exists a level i such that $z_{i,0} + z_{i,1} = 0$, causing the values of δ_i to differ between the evaluators and resulting in $\tau_1^{(0)} \neq \tau_1^{(1)}$.

Security. We show the security of our IVDPF-CDPF in Fig. 5 with the following theorem, and its proof can be found in Appendix E.2.

Theorem 2. Assume the decisional BDH assumption holds in the bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_t$. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}, H' : \{0, 1\}^{n \cdot \lceil \log p \rceil} \rightarrow \{0, 1\}^{2\lambda}$, and $\tilde{H} : \mathbb{G} \rightarrow \{0, 1\}^{2\lambda}$ be collision-resistant hash functions. Let IVDPF be a secure incremental VDPF. The IVDPF-CDPF in Fig. 5 is a secure constrained DPF as described in Definition 1.

6 CDPF with Wildcard-style Constraint

In this section, we propose a DPF scheme with wildcard-style constraints ($\star$ -CDPF) to restrict the function output. We note that, the purpose of previous CDPF schemes is to check whether the dealer is authorized to share a point function with the special input; while $\star$ -CDPF is concerned with whether the secret-shared function is authorized w.r.t its special output. We adopt the verifiable DPF with auxiliary output as we mentioned in Sec. 1. Similar to [32, 33], our CDPF scheme are able to generalize to complex FSS classes derived from DPFs. For general, we consider that each constraint configuration Λ_i includes ℓ entries.

In our $\star$ -CDPF, each authorized set Q_i is represented by ℓ restraint strings $\{\mathbf{rs}_{i,j}\}_{j \in \mathbb{Z}_\ell}$, such that $Q_i := \{f_{i,v} \in \mathcal{F} | \exists j \in \mathbb{Z}_\ell, v \wedge \mathbf{rs}_{i,j} = 0\}$. A point function with special input i matches $\mathbf{rs}_{i,j}$ if and only if all restricted bits of its special output are equal to 0. For clarity, we let $\perp$ be a “valid secret key” for any authorized set Q_i in our $\star$ -CDPF scheme. Note that, our $\star$ -CDPF scheme can be extended to fine-grained constraints by setting each restraint string according to a verification key for a non-trivial secret key.

Fig. 6 presents our $\star$ -CDPF construction for the function family $\mathcal{F} := \{f : \{0, 1\}^n \rightarrow \mathbb{F}_{2^t}\}$. The method **Constraint** sets each constraint configurations to the corresponding restraint string $\mathbf{rs}_{i,j}$. In **FGen**, the dealer generates VDPF keys for $f_{\alpha,\beta} \in \mathcal{F}$ as function shares. Then, the dealer selects the restraint string $\mathbf{rs}_{\alpha,\rho}$ by

the condition $\beta \wedge \mathbf{rs}_{\alpha,\rho} = 0$, and generates a pair of VDPF keys $(f_{\eta,\beta}^{(0)}, f_{\eta,\beta}^{(1)})$ as proof shares, where $\eta := \alpha\ell + \rho$. During the evaluation process, the evaluators first execute the algorithm **VEval** to evaluate function shares and proof shares in VDPF scheme. Then the evaluators generate audit tokens. Concretely, they jointly compute $\llbracket \mathbf{rs}_{\alpha,\rho} \wedge \beta \rrbracket$ by calculating the inner product between the restraint strings $\{\mathbf{rs}_{i,j}\}_{i \in \mathbb{Z}_N, j \in \mathbb{Z}_\ell}$ and the evaluation results of $f_{\eta,\beta}$. Later, each evaluator P_b computes token $\tau_3^{(b)}$ by hashing the share $(\mathbf{rs}_{\alpha,\rho} \wedge \beta)^{(b)}$. In **Verify**, the equality check of $\tau_3^{(0)} = \tau_3^{(1)}$ guarantees $\mathbf{rs}_{\alpha,\rho} \wedge \beta = 0$.

Moreover, we introduce $\tau_0^{(b)}, \tau_1^{(b)}, \tau_2^{(b)}$ for each P_b to prevent the malicious dealer from sharing a mismatched point function $f_{\eta',\beta'}$. The first two tokens $\tau_0^{(b)}, \tau_1^{(b)}$ are VDPF tokens of $f_{\alpha,\beta}^{(b)}, f_{\eta,\beta}^{(b)}$, respectively. They attest the well-formedness of shared point functions. The token $\tau_2^{(b)}$ is computed by hashing the concatenation of the XOR shares of $f_{\alpha,\beta}(i) \oplus (\bigoplus_{j \in \mathbb{Z}_\ell} f_{\eta,\beta}(i \cdot \ell + j))$ for all $i \in \mathbb{Z}_N$. Due to the collision-resistant H , the equality $\tau_2^{(0)} = \tau_2^{(1)}$ in **Verify** holds only if $\eta \in [\alpha\ell, \alpha\ell + \ell)$ and the spacial output of the shared point functions are equal.

Security. We show the security of our $\star$ -CDPF in Fig. 6 with the following theorem, and its proof can be found in Appendix E.3.

Theorem 3. *Let $H : \{0, 1\}^{Nt} \rightarrow \{0, 1\}^{2\lambda}$ be a collision-resistant hash function. Let VDPF be a secure verifiable DPF scheme. The $\star$ -CDPF in Fig. 6 is a secure DPF with constraints as described in Definition 1.*

6.1 Efficiency

In this section, we offer an optimization for CDPF, and discuss the composition of CDPFs for different policies.

DPF tensoring. To support the general $\ell \geq 1$, we introduce a separate VDPF $f_{\eta,(\beta,1)}$ to select the target item in Λ . It results in $O(\lambda \cdot (n + \log \ell))$ communication overhead from the dealer to the evaluators. As observed in prior work [32, 33], the selecting VDPF $f_{\eta,(\beta,1)}$ can reuse the “backbone” of the VDPF $f_{\alpha,\beta}$, which is the DPF tensoring described by Boyle *et al.* [7]. Loosely speaking, the VDPF evaluation result of $f_{\alpha,\beta}$ can be considered as the n -th level results of $f_{\eta,(\beta,1)}$. Therefore, the selecting VDPF only needs to represent a point function with domain size ℓ , and then the communication overhead can be reduce to $O(\lambda \cdot \log \ell)$.

Multi-policy constraint. We observe that, for the same function family $\mathcal{F}$, different CDPF schemes share a common “backbone” of selecting VDPFs. The difference between their selecting VDPFs is the special auxiliary outputs with respect to the different constraints. Due to the privacy of VDPF, all items in the VDPF evaluation result are pseudo-random and independent to each other. Therefore, multiple CDPF schemes for the same $\mathcal{F}$ can form a composite construction for multi-policy constraints using a single selecting VDPF. Here, the selecting point function is converted to a new function with sufficient auxiliary outputs for all CDPFs. The security of the composite construction depends on the security of VDPF and the original CDPF schemes.

7 Implementation and Benchmark

Our CDPF schemes are implemented in Go and C. We implement the underlying DPF followed by [7, 12] and employ the optimization of the FSS tensor technique in [7]. AES-128 is chosen for PRG in DPF, and we implement it by Intel’s AES-NI. We instantiate the bilinear map e in the elliptic curve BLS12-381 [1], and adopt the efficient BLS12-381 implementation in [24]. In addition, we use SHA-256 as the hash function in our constructions.

Our benchmarks are executed on a server with Intel Xeon Silver 4214 CPU at 2.20GHz running Ubuntu 20.04.5 LTS; with 48 vCPUs and 128 GB Memory. Each experiment result is an average from 500 - 1000 evaluations. For the purpose of comparison, we also perform the same benchmarks for the DPF-PACL and VDPF-PACL from [32, 33]. We rerun the source code [31] provided by the original authors, where the DPF-PACL uses the group $\mathbb{G}$ as the P-256 elliptic curve group, and the VDPF-PACL uses the group $\mathbb{G} := \mathbb{Z}_p^*$ with

a 3072-bit prime p as specified in RFC3526 [23]. In addition, we evaluate a CDPF schemes for multi-policy constraints: the SK- $\star$ -CDPF composed of the VDPF-PACL and our $\star$ -CDPF.

Parameters. Set $\lambda = 128$ and $n = 32$. In practical, such as mailbox system [14], constraints are applied to a registered set X , and the evaluators evaluate DPF on X rather than the full domain. Denote $N = |X|$ as the number of constraint configurations, and ℓ as the number of entries per configuration.

Optimizations. We reduce the communication cost between the evaluators by XOR-accumulating some audit tokens, which are just verified by an equality check in `Verify`.

7.1 Proof time

Table 1 compares the proof time of schemes. That is, the running time of `Prove` in the DPF-PACL and VDPF-PACL; and the running time of `FGen` except the function share generation in our schemes. We note that, using the FSS tensor optimization, the computational complexity of proof time is log-linear in ℓ . The DPF-PACL and our $\star$ -CDPF schemes achieve an overwhelming performance advantage, because their operations are minimal except for (V)DPF key generation. Our IVDPF-CDPF scheme requires $O(n)$ group operations for layer outputs in the IVDPF key generation, resulting in additional running time. For the SK- $\star$ -CDPF scheme, multiple constraints share a common selecting point function to save a VDPF generation. Due to the succinct of the underlying DPF representation, this optimization appears insignificant in the proof generation.

Table 1: Performance of proof generation.

# of entries per Λ_i	1	2^5	2^{10}	2^{15}
DPF-PACL [32, 33]	0.3 μ s	1.37 μ s	2.74 μ s	3.13 μ s
VDPF-PACL [32, 33]	30.1 ms	29.3 ms	29.9 ms	29.3 ms
IVDPF-CDPF (Ours)	31.7 μ s	41.6 μ s	50.2 μ s	63.8 μ s
$\star$ -CDPF (Ours)	0.6 μ s	1.49 μ s	2.65 μ s	3.05 μ s
SK- $\star$ -CDPF	29.1 ms	30.2 ms	29.4 ms	29.3 ms

7.2 Communication overhead

There are two communication rounds in our CDPF schemes and PACLs: one is that the dealer distributes the function shares and proof shares to the evaluators; the other is that two evaluators exchange their audit tokens. Table 2 reports the concrete communication overhead of the two rounds. Let s_ℓ be the standard DPF key size for the point functions with domain size ℓ . The VDPF key size of [12] is $s_\ell + 64$ bytes, and our IVDPF key size is $s_\ell + 96 \log \ell$ bytes due to $\log \ell$ correction seeds and layer correction words.

Table 2: Proof and audit token sizes in bytes, where s_ℓ is the standard DPF key size for the domain size ℓ .

	Proof Size	Token Size
DPF-PACL [32, 33]	$32 + s_\ell$	64
VDPF-PACL [32, 33]	$1952 + s_\ell + 64$	816
Fixed VDPF-PACL	$1952 + s_\ell + 64$	880
IVDPF-CDPF (Ours)	$385 + s_\ell + 96 \log \ell$	64
$\star$ -CDPF (Ours)	$s_\ell + 64$	64
SK- $\star$ -CDPF	$1952 + s_\ell + 64$	880

The proof size and token size of our $\star$ -CDPF and IVDPF-CDPF schemes are smaller than those of VDPF-PACL. Compared to the sum of multiple CDPFs, the SK- $\star$ -CDPF scheme saves one VDPF key in proof and save one hash value in audit token. In particular, we compare our fixed VDPF-PACL with the VDPF-PACL in [32, 33]. As shown in Table 2, our security improvement is almost free, in fact it only increases the token size by 64 bytes.

7.3 Verification cost

The evaluators execute VEval (or Audit in PACLs) and Verify sequentially for verification. Thus, we take the total run-time of the two algorithm as the verification cost. To demonstrate the advantage of our IVDPF-CDPF scheme, we plot the run-time of verification by steps in Fig. 7, where $\ell = 1$. It shows that, for both DPF-PACL and VDPF-PACL, the most costly step is selecting the target verification key. Because they use N verification keys to represent N constraint configurations, and require $O(N)$ exponentiation/multiplication operations in the key selection step. Our IVDPF-CDPF reduces the constraint representation to $2 \log N$ public keys, so that only $O(\log N)$ exponentiation operations are required. Thus, our IVDPF-CDPF scheme is significantly more efficient than PACLs when N is large. For instance, our IVDPF-CDPF improves the performance of DPF-PACL by a factor of $2.5\times$ for 2^{15} constraint configurations, and it is $2\times$ faster than VDPF-PACL.

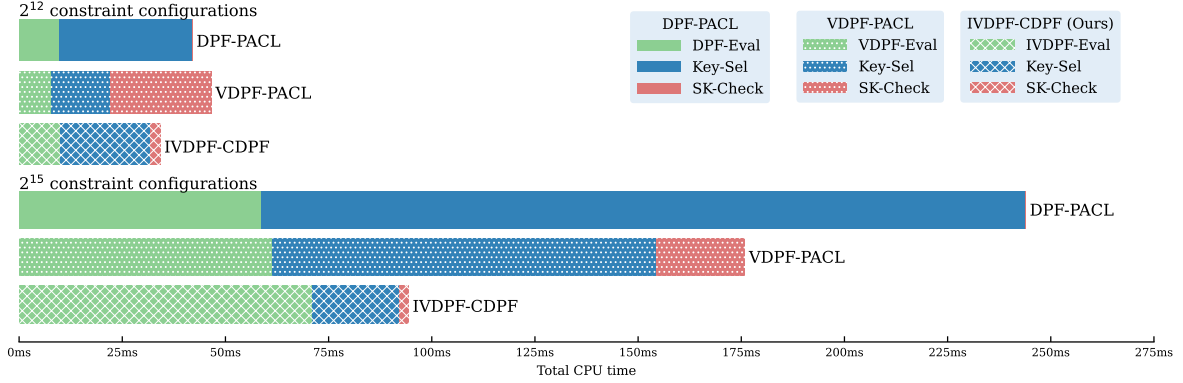


Figure 7: Verification performance of PACLs [32, 33] and our IVDPF-CDPF scheme.

In addition, we compare our $\star$ -CDPF and the VDPF-PACL in Fig. 8, while the VDPF evaluation of $f_{\alpha, \beta}$ is the baseline. As the number of constraint configurations N increases, the amortized overhead of VDPF-PACL decreases. This is because the overhead of the SPoSS and the equality check are amortized over N . On the contrary, the amortized overhead of our $\star$ -CDPF plateaus at the beginning due to its minimal operations. For the SK- $\star$ -CDPF, its verification time is less than the sum of corresponding original schemes because it halves VDPF evaluations. This advantage would be significant when the input domain is large.

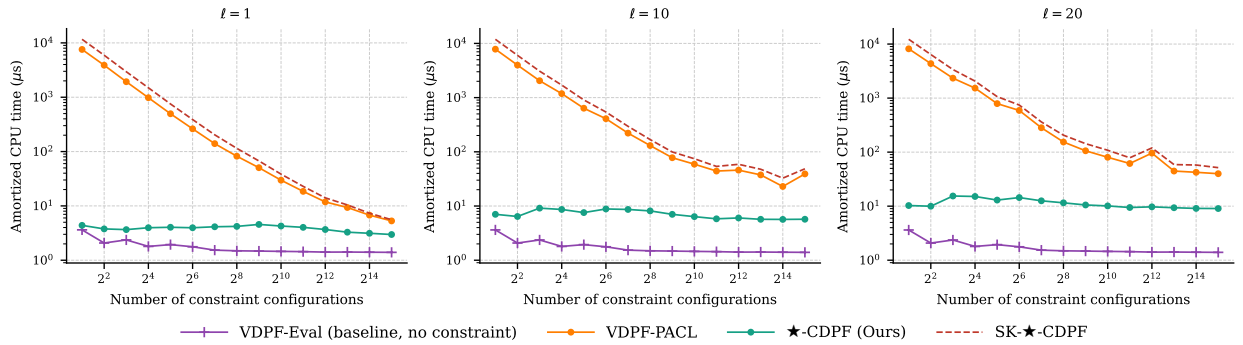


Figure 8: Comparison of the amortized verification cost, where VDPF-PACL refers to [32, 33]. The run-time is amortized by the number of constraint configurations N .

8 CDPF with Non-interactive Evaluators: Impossibility and Feasibility

To the best of our knowledge, all known CDPF constructions require the evaluators to have at least one round of communication with each other before the decision could be made. A natural question is whether there exists a CDPF scheme in which the evaluators do not need to communicate with each other. We call such type of CDPF schemes as “non-interactive CDPF” and denote it as ni-CDPF. In this section, we provide the impossibility and feasibility results of ni-CDPF.

Impossibility. We show that it is impossible to construct a secure ni-CDPF in the plain model. Formally,

Theorem 4. *There exists no ni-CDPF construction in the plain model that is a secure CDPF as described in Definition 1.*

Proof. We now prove the theorem by contradiction. Assume there exists a secure CDPF with non-interactive evaluators in the plain model, denoted as $(\text{Constraint}, \text{SKGen}, \text{FGen}, \text{V Eval}, \text{Verify})$, for a function family $\mathcal{F} := \{f : \mathcal{D} \rightarrow \mathbb{G}\}$. That is, the verification result of P_b only depends on a single token $\tau^{(b)} \leftarrow \text{Audit}(b, f^{(b)}, \pi^{(b)})$. Due to the privacy definition of CDPF, there is a PPT simulator Sim such that

$$\text{View}_{\text{CDPF}}(b, \Lambda, f, \text{sk}, X) \approx_c \text{Sim}(1^\lambda, b, \Lambda, X).$$

Now, we construct an efficient adversary $\mathcal{A}$ using Sim . On input $(1^\lambda, \Lambda)$, the adversary $\mathcal{A}$ proceeds as follows:

- 1 Pick a random subset X of $\mathcal{D}$;
- 2 $(f^{(0)}, \pi^{(0)}) \leftarrow \text{Sim}(1^\lambda, 0, \Lambda, X)$;
- 3 $(f^{(1)}, \pi^{(1)}) \leftarrow \text{Sim}(1^\lambda, 1, \Lambda, X)$;
- 4 Output $((f^{(0)}, f^{(1)}), (\pi^{(0)}, \pi^{(1)}), X)$.

According to the privacy definition, for any $b \in \{0, 1\}$, the output $(f^{(b)}, \pi^{(b)})$ of Sim passes the verification of P_b . Hence, $\mathcal{A}$ can win the soundness game with noticeable probability, which contradicts the security of the scheme. $\square$

Feasibility. Our impossibility result above is based on the fact that if the evaluators do not communicate with each other, then they cannot coordinate with each other in the plain model. If the necessary “coordination” between the evaluators can be enabled, say through certain trusted setups, then we may be able to securely remove the “online” communication between the evaluators. If this is true, then we essentially obtain the first non-interactive CDPF!

Based on the above idea, we now describe a generic compiler for ni-CDPF using non-interactive zero-knowledge proofs (NIZKs)⁶, PKI, and a broadcast channel. Let $(\text{KGen}, \text{Enc}, \text{Dec})$ denote the public-key encryption scheme. Each P_b generates a public-private key pair $(\text{pk}_b, \text{sk}_b) \leftarrow \text{KGen}(1^\lambda)$, and then registers the public key pk_b to the PKI. The dealer obtains valid public keys pk_0, pk_1 from the PKI.

Given any secure CDPF scheme with one-round interactive evaluators, denoted as $\text{CDPF} := (\text{Constraint}, \text{SKGen}, \text{FGen}, \text{V Eval}, \text{Verify})$, we can eliminate the interaction by letting the dealer broadcast the encrypted function shares, the corresponding tokens and an NIZK proof. More specifically, the dealer D first generates function shares with proof shares $((f^{(0)}, \pi^{(0)}), (f^{(1)}, \pi^{(1)}))$ by FGen following the CDPF. Next, for each $b \in \{0, 1\}$, D encrypts the function share $f^{(b)}$ with a randomness $r^{(b)}$ as $C^{(b)} := \text{Enc}(\text{pk}_b, (f^{(b)}, \pi^{(b)}), r^{(b)})$, then broadcasts the ciphertext $C^{(b)}$. For non-interactive verification, we let D compute and broadcast the token $\tau^{(b)}$ from $(Y^{(b)}, \tau^{(b)}) := \text{V Eval}(b, \Lambda, f^{(b)}, \pi^{(b)}, X)$ for each $b \in \{0, 1\}$, where Λ is the public constraint configuration list and X is the evaluation input set in the CDPF. If the dealer D provides correct tokens, each evaluator P_b can locally check if $\text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1$ to make a decision.

To prevent a malicious dealer D from forging $\tau^{(0)}, \tau^{(1)}$, we require the dealer D to provide an NIZK proof $\tilde{\pi}$ to prove the validity of tokens. Concretely, the statement consists of the ciphertexts and tokens being broadcast, the public keys, the configurations, and the input set, denoted as $\text{st} := (C^{(0)}, \tau^{(0)}, C^{(1)}, \tau^{(1)}, \text{pk}_0, \text{pk}_1, \Lambda, X)$. The witness includes the function shares, the proof shares, and the randomness D uses in the encryption,

⁶Note that, NIZK is usually realized in the random oracle and/or common reference string (CRS) models.

denoted as $w := (f^{(0)}, \pi^{(0)}, r^{(0)}, f^{(1)}, \pi^{(1)}, r^{(1)})$. The NIZK proof $\tilde{\pi}$ is for the efficiently decidable binary relation

$$R := \left\{ (\text{st}, w) \left| \begin{array}{l} C^{(0)} = \text{Enc}(\text{pk}_0, (f^{(0)}, \pi^{(0)}, r^{(0)})) \wedge \\ C^{(1)} = \text{Enc}(\text{pk}_1, (f^{(1)}, \pi^{(1)}, r^{(1)})) \wedge \\ ((Y^{(0)}, t^{(0)}) := \text{VEval}(0, \Lambda, f^{(0)}, \pi^{(0)}, X), \tau^{(0)} = t^{(0)}) \wedge \\ ((Y^{(1)}, t^{(1)}) := \text{VEval}(1, \Lambda, f^{(1)}, \pi^{(1)}, X), \tau^{(1)} = t^{(1)}) \end{array} \right. \right\}$$

Each P_b can locally verify the NIZK proof $\tilde{\pi}$ to check the validity of the tokens.

If the verification of $\tilde{\pi}$ passes and $\text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1$, P_b decrypts and accepts the function share $(f^{(b)}, \pi^{(b)}) := \text{Dec}(\text{sk}_b, C^{(b)})$, then evaluate the function share by executing $\text{VEval}(b, \Lambda, f^{(b)}, \pi^{(b)}, X)$; otherwise, P_b rejects the function share.

We remark that, our purpose here is to illustrate the main ideas for constructing a non-interactive CDPF ni-CDPF scheme. When practical alternatives for NIZK are available, we indeed are able to obtain practical ni-CDPF schemes. In the Appendix D, we will propose a practical construction in the preprocessing model. Its verification is based on the *Vector Oblivious Linear Evaluation* (VOLE) correlations. As a subtle drawback of this scheme, it cannot ensure the honest evaluators reach consensus. In particular, a malicious dealer without valid secret key can successfully cheat one of the evaluators. Nevertheless, our practical ni-CDPF scheme can be securely used for private information retrieval applications [19, 35]. In these applications, the drawback of our scheme can be easily addressed by re-randomizing servers' (i.e., the CDPF evaluators) responses. Denote the original response of P_b as $\text{res}^{(b)}$. Suppose servers agree on a random seed s in the initialization phase. If the server P_b accepts the function share, P_b responds to the user (i.e., the CDPF dealer) with re-randomized $\text{res}^{(b)} + (-1)^b \cdot G(s)$ where G is a PRG, rather than the original value; otherwise, the server responds with $\perp$. Clearly, the user cannot learn any information of data from a single valid response.

References

- [1] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. Constructing elliptic curves with prescribed embedding degrees. In Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, editors, *SCN 02*, volume 2576 of *LNCS*, pages 257–267, Amalfi, Italy, September 12–13, 2003. Springer Berlin Heidelberg, Germany.
- [2] Amos Beimel. *Secure Schemes for Secret Sharing and Key Distribution*. PhD thesis, Israel Institute of Technology, Technion, Haifa, Israel, 1996.
- [3] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Lightweight techniques for private heavy hitters. In *2021 IEEE Symposium on Security and Privacy*, pages 762–776, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.
- [4] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 213–229, Santa Barbara, CA, USA, August 19–23, 2001. Springer Berlin Heidelberg, Germany.
- [5] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 871–900, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- [6] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 337–367, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany.
- [7] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1292–1303, Vienna, Austria, October 24–28, 2016. ACM Press.

- [8] Paul Bunn, Jonathan Katz, Eyal Kushilevitz, and Rafail Ostrovsky. Efficient 3-party distributed ORAM. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 215–232, Amalfi, Italy, September 14–16, 2020. Springer, Cham, Switzerland.
- [9] Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *2015 IEEE Symposium on Security and Privacy*, pages 321–338, San Jose, CA, USA, May 17–21, 2015. IEEE Computer Society Press.
- [10] Emma Dauterman, Mayank Rathee, Raluca Ada Popa, and Ion Stoica. Waldo: A private time-series database from function secret sharing. In *2022 IEEE Symposium on Security and Privacy*, pages 2450–2468, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press.
- [11] Hannah Davis, Christopher Patton, Mike Rosulek, and Phillipp Schoppmann. Verifiable distributed aggregation functions. *Proceedings on Privacy Enhancing Technologies*, 2023.
- [12] Leo de Castro and Antigoni Polychroniadou. Lightweight, maliciously secure verifiable function secret sharing. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 150–179, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- [13] Jack Doerner and abhi shelat. Scaling ORAM for secure computation. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 523–535, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- [14] Saba Eskandarian, Henry Corrigan-Gibbs, Matei Zaharia, and Dan Boneh. Express: Lowering the cost of metadata-hiding communication with cryptographic privacy. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 1775–1792. USENIX Association, August 11–13, 2021.
- [15] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 640–658, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- [16] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *25th FOCS*, pages 464–479, Singer Island, Florida, October 24–26, 1984. IEEE Computer Society Press.
- [17] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 89–98, Alexandria, Virginia, USA, October 30 – November 3, 2006. ACM Press. Available as Cryptology ePrint Archive Report 2006/309.
- [18] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. SIGMA: Secure GPT inference with function secret sharing. *PoPETs*, 2024(4):61–79, October 2024.
- [19] Trinabh Gupta, Natacha Crooks, Whitney Mulhern, Srinath Setty, Lorenzo Alvisi, and Michael Walfish. Scalable and private media consumption with popcorn. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, pages 91–107, Santa Clara, CA, March 2016. USENIX Association.
- [20] Keyu Ji, Bingsheng Zhang, Tianpei Lu, and Kui Ren. Multi-party private function evaluation for ram. *IEEE Transactions on Information Forensics and Security*, 18:1252–1267, 2023.
- [21] Keyu Ji, Bingsheng Zhang, and Kui Ren. Fine-grained policy constraints for distributed point function. Cryptology ePrint Archive, Paper 2023/1672, 2023. <https://eprint.iacr.org/2023/1672>.
- [22] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Chapman and Hall/CRC, New York, 3rd edition, 2020. eBook Published on December 21, 2020.
- [23] Mika Kojo and Tero Kivinen. More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE). RFC 3526, 2003.

- [24] Sam Kumar, Yuncong Hu, Michael P. Andersen, Raluca Ada Popa, and David E. Culler. JEDI: Many-to-many end-to-end encryption and key delegation for IoT. In Nadia Heninger and Patrick Traynor, editors, *USENIX Security 2019*, pages 1519–1536, Santa Clara, CA, USA, August 14–16, 2019. USENIX Association.
- [25] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. Delphi: A cryptographic inference service for neural networks. In Srdjan Capkun and Franziska Roesner, editors, *USENIX Security 2020*, pages 2505–2522. USENIX Association, August 12–14, 2020.
- [26] Dimitris Mouris, Pratik Sarkar, and Nektarios Georgios Tsoutsos. PLASMA: Private, lightweight aggregated statistics against malicious adversaries. *PoPETs*, 2024(3):4–24, July 2024.
- [27] Dimitris Mouris, Pratik Sarkar, and Nektarios Georgios Tsoutsos. Plasma: Private, lightweight aggregated statistics against malicious adversaries. *Proceedings on Privacy Enhancing Technologies*, 2024.
- [28] Zachary Newman, Sacha Servan-Schreiber, and Srinivas Devadas. Spectrum: High-bandwidth anonymous broadcast. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 229–248, Renton, WA, April 2022. USENIX Association.
- [29] Margaret Pierce and Saba Eskandarian. Silent splitter: Privacy for payment splitting via new protocols for distributed point functions. *PoPETs*, 2025(4):996–1013, October 2025.
- [30] Théo Ryffel, Pierre Tholoniati, David Pointcheval, and Francis R. Bach. AriaNN: Low-interaction privacy-preserving deep learning via function secret sharing. *PoPETs*, 2022(1):291–316, January 2022.
- [31] Sacha Servan-Schreiber, Simon Beyzerov, Eli Yablon, and Hyojae Park. PACL source code. github, 2022. <https://github.com/sachaservan/pacl>.
- [32] Sacha Servan-Schreiber, Simon Beyzerov, Eli Yablon, and Hyojae Park. Private access control for function secret sharing. Cryptology ePrint Archive, Report 2022/1707, 2022.
- [33] Sacha Servan-Schreiber, Simon Beyzerov, Eli Yablon, and Hyojae Park. Private access control for function secret sharing. In *2023 IEEE Symposium on Security and Privacy*, pages 809–828, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press.
- [34] Adithya Vadapalli, Kyle Storrier, and Ryan Henry. Sabre: Sender-anonymous messaging with fast audits. In *2022 IEEE Symposium on Security and Privacy*, pages 1953–1970, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press.
- [35] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. Splinter: Practical private queries on public data. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 299–313, Boston, MA, March 2017. USENIX Association.
- [36] Mark N. Wegman and Larry Carter. New hash functions and their use in authentication and set equality. *Journal of Computer and System Sciences*, 22:265–279, 1981.
- [37] Zhelei Zhou, Bingsheng Zhang, Hong-Sheng Zhou, and Kui Ren. Single-input functionality against a dishonest majority: Practical and round-optimal. Cryptology ePrint Archive, Paper 2024/305, 2024.

A Definition of VDPF

In this section, we recap the VDPF definition from [12].

Definition 3 (Verifiable Distributed Point Function [12]). *A 2-party VDPF scheme, parameterized by a function family $\mathcal{F}$ of point functions $f : \{0, 1\}^n \rightarrow \mathbb{G}$, consists of three PPT algorithms (Gen, Eval, Verify) defined as follows:*

- $(f_{\alpha, \beta}^{(0)}, f_{\alpha, \beta}^{(1)}) \leftarrow \text{Gen}(1^\lambda, f_{\alpha, \beta})$ is the share generation algorithm that takes input as the security parameter 1^λ and a point function $f_{\alpha, \beta} \in \mathcal{F}$. It outputs a pair of VDPF keys, i.e., the additive shares of $\llbracket f_{\alpha, \beta} \rrbracket$.

- $(\{y_x^{(b)}\}_{x \in X}, \tau^{(b)}) \leftarrow \text{Eval}(b, f_{\alpha, \beta}^{(b)}, X)$ is the verifiable evaluation algorithm that takes input as an index $b \in \{0, 1\}$, a VDPF key $f_{\alpha, \beta}^{(b)}$, and a set of function inputs $X \subseteq \{0, 1\}^n$. It outputs a tuple of values. The first set of values are the FSS outputs, which are additive shares of $f_{\alpha, \beta}(x), x \in X$. The second item is a token $\tau^{(b)}$ that is used to verify the well-formedness of the shared function.
- $1 \text{ or } 0 \leftarrow \text{Verify}(\tau^{(0)}, \tau^{(1)})$ is the verification algorithm that takes input as a pair of tokens. It outputs 1 for acceptance or 0 for rejection.

A secure VDPF must satisfy three properties as follows:

- **Completeness.** For all $f \in \mathcal{F}$, $X \subseteq \{0, 1\}^n$, it holds that

$$\Pr \left[\begin{array}{l} (f^{(0)}, f^{(1)}) \leftarrow \text{Gen}(1^\lambda, f); \\ (\{y_x^{(0)}\}, \tau^{(0)}) \leftarrow \text{Eval}(0, f^{(0)}, X); \\ (\{y_x^{(1)}\}, \tau^{(1)}) \leftarrow \text{Eval}(1, f^{(1)}, X); \\ \forall x \in X, y_x^{(0)} + y_x^{(1)} = f(x) \wedge \\ \text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1 \end{array} \right] = 1$$

- **Privacy.** For a function $f \in \mathcal{F}$ and a set of inputs $X \subseteq \{0, 1\}^n$, define the view of evaluator P_b $\text{View}_{\text{VDPF}}(b, f, X)$ as the probability distribution ensemble $\{(f^{(b)}, \tau^{(1-b)})\}_\lambda$, where $(f^{(0)}, f^{(1)}) \leftarrow \text{Gen}(1^\lambda, f)$, $(\{y_x^{(1-b)}\}_{x \in X}, \tau^{(1-b)}) \leftarrow \text{Eval}(1 - b, f^{(1-b)}, X)$. There exists a PPT simulator Sim such that for all $f \in \mathcal{F}, X \subseteq \{0, 1\}^n$, we have:

$$\text{View}_{\text{VDPF}}(b, f, X) \approx_c \text{Sim}(1^\lambda, b, n, \mathbb{G}, X)$$

- **Soundness.** Let $f^{(b)}$ be the (possibly maliciously generated) share received by the evaluator P_b . There exists a negligible function negl such that for any $X \subseteq \{0, 1\}^n$, $(\{y_x^{(b)}\}_{x \in X}, \tau^{(b)}) \leftarrow \text{Eval}(b, f^{(b)}, X)$ for $b \in \{0, 1\}$, it holds that

$$\Pr \left[\begin{array}{l} \text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1 : \\ \left| \{x \in X \mid y_x^{(0)} + y_x^{(1)} \neq 0\} \right| \leq 1 \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

B Definition of PACL

For simplicity, we focus on 2-party FSS scheme, i.e. (2, 2)-FSS, and then recap the PACL definition in [33]:

Definition 4. Let integer $N := 2^n$ and $\mathcal{F} := \{f_i : \{0, 1\}^n \rightarrow \{0, 1\}^* \mid 1 \leq i \leq N\}$ be a family of functions. Let $(\text{Gen}, \text{Eval})$ instantiate a 2-party FSS scheme for $\mathcal{F}$. A PACL scheme consists of efficient algorithms KeyGen , Prove , Audit and Verify defined as follows:

- $(\text{vk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda, f)$ takes as input a security parameter 1^λ and a function $f \in \mathcal{F}$, and outputs a new pair of verification and access keys (vk, sk) .
- $(\pi^{(0)}, \pi^{(1)}) \leftarrow \text{Prove}(f, \text{sk})$ takes as input a function $f \in \mathcal{F}$ and access key sk , and outputs a vector of 2 proof secret shares $(\pi^{(0)}, \pi^{(1)})$.
- $\tau^{(b)} \leftarrow \text{Audit}(b, \Lambda, f^{(b)}, \pi^{(b)})$ takes as input access control list $\Lambda = (\text{vk}_1, \dots, \text{vk}_N)$, function secret share $f^{(b)}$ of f sampled according to Gen , and proof share $\pi^{(b)}$, and outputs audit token $\tau^{(b)}$.
- $1 \text{ or } 0 \leftarrow \text{Verify}(\tau^{(0)}, \tau^{(1)})$ takes as input a set of 2 audit tokens, and outputs 1 for acceptance or 0 for rejection.

The above functionality must satisfy:

- **Completeness.** A PACL scheme is complete if for all security parameters λ , for all $b \in \{0, 1\}$, for all $\Lambda := (\text{vk}_1, \dots, \text{vk}_N)$ where $\forall i, \text{vk}_i$ is sampled according to KeyGen , and for all secret shares $(f^{(0)}, f^{(1)})$ of $f \in \mathcal{F}$ sampled according to $\text{Gen}(1^\lambda, f)$, it holds that

$$\Pr \left[\begin{array}{l} (\pi^{(0)}, \pi^{(1)}) \leftarrow \text{Prove}(f, \text{sk}); \\ (f^{(0)}, f^{(1)}) \leftarrow \text{Gen}(1^\lambda, f); \\ \{\tau^{(b)} \leftarrow \text{Audit}(b, \Lambda, f^{(b)}, \pi^{(b)}) \mid b \in \{0, 1\}\} : \\ \text{Verify}(\tau^{(0)}, \tau^{(1)}) = \text{Check}(\Lambda, f, \text{sk}) \end{array} \right] = 1$$

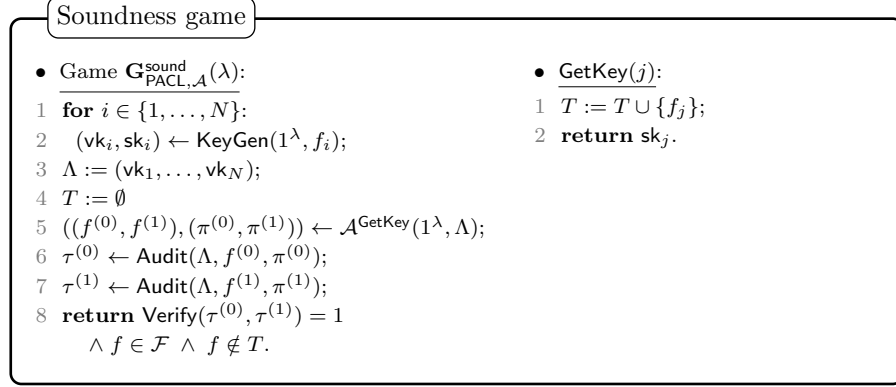


Figure 9: Soundness game for the PACL scheme $\text{PACL} = (\text{KeyGen}, \text{Prove}, \text{Audit}, \text{Verify})$ for the function family $\mathcal{F}$.

- **Efficiency.** The size of each proof share $\pi^{(b)}$ is most $O(\lambda N^\epsilon)$ for any $\epsilon < 1$ (possibly dependent on n). The size of each audit token $\tau^{(b)}$ is $O(\lambda)$.
- **Privacy.** For any $b \in \{0, 1\}$, define View_b to be the distribution over $\{(\pi^{(b)}, \tau^{*(b)})\} \cup \{\tau^{(1-b)}\}$ where $\pi^{(b)}$ is sampled according to $\text{Prove}(f, \text{sk})$, $\tau^{*(b)}$ is sampled arbitrarily and $\tau^{(1-b)}$ is generated by $\text{Audit}(\Lambda, f^{(1-b)}, \pi^{(1-b)})$. A PACL is private if there exists an efficient simulator Sim such that

$$\text{View}_b \approx_c \text{Sim}(1^\lambda, b, \Lambda, \tau^{*(b)}).$$

That is, the distribution of proof shares and audit shares reveal nothing about f_i or the access key sk to any one computationally bounded (possibly malicious) verifier.

- **Soundness.** For any PPT adversary $\mathcal{A}$ with oracle access to GetKey , it holds that

$$\Pr[\mathbf{G}_{\text{PACL}, \mathcal{A}}^{\text{sound}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

where $\mathbf{G}_{\text{PACL}, \mathcal{A}}^{\text{sound}}(\lambda)$ is depicted in Fig. 9.

First, notice that the scheme allows simultaneous messaging, i.e., all the parties can send messages to each other in the same round. Following the simulation paradigm, typically, rushing adversary model is considered; that is, the adversary's messages can be generated at the end of the round, after seeing the honest parties' messages. However, in the privacy definition of [33], the simulator is given the adversary's message as input, which deviates from the reality.

Second, during the real execution, each evaluator receives a function share and a proof share from the dealer as well as an audit token from the other evaluator. However, in the privacy definition of [33], the view of evaluator does not include the function share. We show a counterexample to the claim that this privacy definition is problematic. Given any secure $\text{PACL} := (\text{KeyGen}, \text{Prove}, \text{Audit}, \text{Verify})$ of a 2-party VDPF for a point function family $\mathcal{F} := \{f : \{0, 1\}^n \rightarrow \mathbb{G}\}$ under their definition, we construct a new audit algorithm $\text{Audit}'(\Lambda, f^{(b)}, \pi^{(b)})$ that outputs $\tau'^{(b)} := (\tau^{(b)} \leftarrow \text{Audit}(\Lambda, f^{(b)}, \pi^{(b)}), f^{(b)})$, and then obtain a new PACL scheme $\text{PACL}' := (\text{KeyGen}, \text{Prove}, \text{Audit}', \text{Verify})$. Easy to note that, PACL' leaks the secret f . Unfortunately, we show that it is a secure PACL under their definition. Obviously, PACL' satisfies the completeness and soundness property due to the security of PACL. For the privacy of this PACL, the view of $\mathcal{P}_b$ is $\{(\pi^{(b)}, \tau^{*(b)})\} \cup \{\tau'^{(1-b)}\}$ where $\tau'^{(1-b)} \leftarrow \text{Audit}'(\Lambda, f^{(1-b)}, \pi^{(1-b)})$. Denote the efficient simulator of PACL as Sim_{PACL} , the efficient simulator of VDPF as Sim_{VDPF} . We construct an efficient simulator Sim' for PACL' as follows: On input $(1^\lambda, b, \Lambda, \tau^{*(b)})$, Sim' proceeds

- 1 $(\pi^{(b)}, \tau^{*(b)}, \tau^{(1-b)}) \leftarrow \text{Sim}_{\text{PACL}}(1^\lambda, b, \Lambda, \tau^{*(b)})$;
- 2 $(f^{(1-b)}, -) \leftarrow \text{Sim}_{\text{VDPF}}(1^\lambda, 1-b, \mathcal{F}, \mathbb{Z}_N)$;
- 3 Output $(\pi^{(b)}, \tau^{*(b)}, (\tau^{(1-b)}, f^{(1-b)}))$.

Due to the privacy of PACL and VDPF, the output distribution of Sim is computationally indistinguishable from the distribution of real view of P_b . Therefore, PACL' indeed leaks the function f but is a secure scheme under the definition of [33]. In addition, to satisfy the efficiency property that the audit token size is $O(\lambda)$, we can make Audit' replace $f^{(b)}$ in the output with $y_1^{(b)} \in \mathbb{G}$ generated by $\text{VDPF.Eval}(b, f^{(b)}, \{1\})$, which still leaks some information of the secret f .

C Properties of Definiton 1 for the Special Case

If the verification result of P_b only depends on $\tau^{(b)}$, the security properties of Definiton 1 are defined as follows:

- **Completeness.** For any $\{(\mathcal{Q}_i, \mathbb{A}_i)\}_{i \in \mathbb{Z}_N}$ such that $\text{Constraint}(1^\lambda, \{(\mathcal{Q}_i, \mathbb{A}_i)\}_{i \in \mathbb{Z}_N}) \rightarrow (\Lambda, \text{msk}) \neq \perp$, any $i \in \mathbb{Z}_N$, any function $f \in \mathcal{Q}_i$, any secret key $\text{sk} \leftarrow \text{SKGen}(1^\lambda, \text{id}, \text{msk})$ where id belongs to $\mathbb{A}_i$, and any $X \subseteq \mathcal{D}$, it holds that

$$\Pr \left[\begin{array}{l} ((f^{(0)}, \pi^{(0)}), (f^{(1)}, \pi^{(1)})) \leftarrow \text{FGen}(\Lambda, f, \text{sk}); \\ (\{y_x^{(0)}\}_{x \in X}, \tau^{(0)}) \leftarrow \text{VEval}(0, \Lambda, f^{(0)}, \pi^{(0)}, X); \\ (\{y_x^{(1)}\}_{x \in X}, \tau^{(1)}) \leftarrow \text{VEval}(1, \Lambda, f^{(1)}, \pi^{(1)}, X); \\ \forall x \in X, y_x^{(0)} + y_x^{(1)} = f(x) \wedge \\ \text{Verify}(\tau^{(0)}, \perp) = 1 \wedge \text{Verify}(\tau^{(1)}, \perp) = 1 \end{array} \right] = 1$$

- **Privacy.** For a constraint configuration list Λ , a function $f \in \mathcal{F}$, a secret key sk and a set of function inputs $X \subseteq \mathcal{D}$, define the distribution representing the view of P_b as

$$\text{View}_{\text{CDPF}}(b, \Lambda, f, \text{sk}, X) := \{(f^{(b)}, \pi^{(b)})\}$$

where $((f^{(0)}, \pi^{(0)}), (f^{(1)}, \pi^{(1)})) \leftarrow \text{FGen}(\Lambda, f, \text{sk})$.

There exists a PPT simulator Sim such that for any Λ, msk generated from $\text{Constraint}(1^\lambda, \{(\mathcal{Q}_i, \mathbb{A}_i)\}_{i \in \mathbb{Z}_N})$ for any $\{(\mathcal{Q}_i, \mathbb{A}_i)\}_{i \in \mathbb{Z}_N}$, any function f from any $\mathcal{Q}_i$, any secret key $\text{sk} \leftarrow \text{SKGen}(1^\lambda, \text{id}, \text{msk})$ where id belongs to $\mathbb{A}_i$, any $X \subseteq \mathcal{D}$, and any $b \in \{0, 1\}$, the following two distributions are computationally indistinguishable:

$$\text{View}_{\text{CDPF}}(b, \Lambda, f, \text{sk}, X) \approx_c \text{Sim}(1^\lambda, b, \Lambda, X)$$

- **Soundness.** For any PPT adversary $\mathcal{A}$, it holds that

$$\Pr[\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

where the game $\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}(\lambda)$ is depicted in Fig. 10.

D CDPF with Non-Interactive Evaluators in the Preprocessing Model

In this section, we propose a practical ni-CDPF scheme, i.e., CDPF with non-interactive evaluators, in the preprocessing model. We assume all parties are connected by pairwise secure channels and a broadcast channel.

For readability, we present our ni-CDPF scheme as a protocol $\Pi_{\text{ni-CDPF}}$ in Fig. 11. It is based on the VDPF technique from [12] and multi-verifier (subfield) VOLE from [37]. Similar to the VDPF-PACL in [32, 33], our constraint configuration list $\Lambda := \{\Lambda_i\}$ is a series of group elements $\Lambda_i \in \mathbb{G} := \mathbb{Z}_p^*$, named verification keys. Let g be the generator of $\mathbb{G}$. The discrete logarithm of each verification key Λ_i to the base g is the secret key of Λ_i . Therefore, given a shared point function with special input α , the evaluators check if the dealer knows the correct discrete logarithm of Λ_α to make a decision.

As mentioned in Sec. 1, we first need to ensure that the dealer D distributes a pair of well-formed VDPF keys for a point function $f_{\alpha, \beta}$ with an auxiliary output 1. Then the evaluators can obviously obtain the shared valid verification key $\langle \Lambda_\alpha \rangle$ over $\mathbb{Z}_p$. Naively, the verification of VDPF keys requires one-round

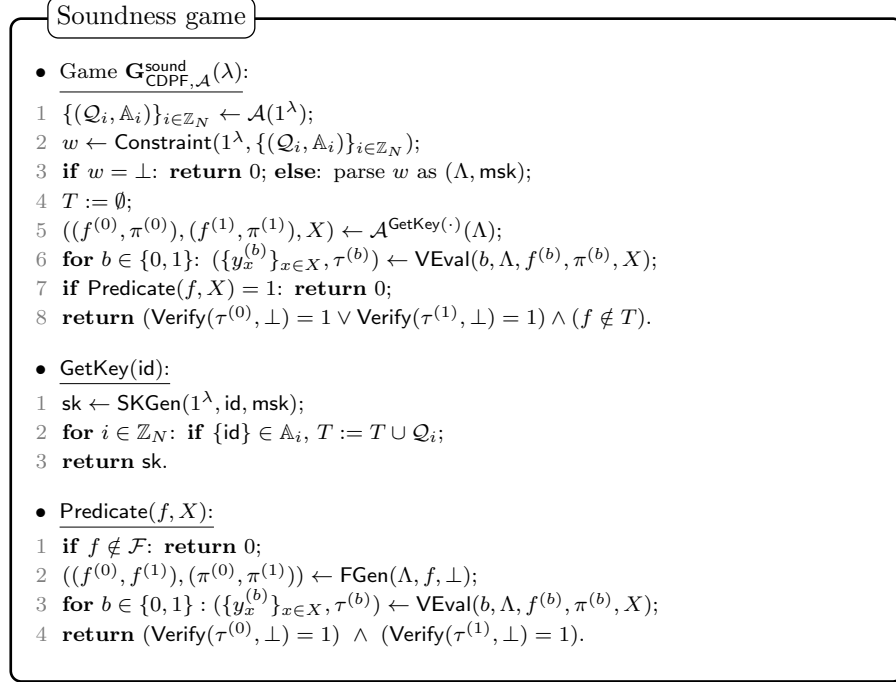


Figure 10: Soundness game for constrained distributed point function $\text{CDPF} = (\text{Constraint}, \text{SKGen}, \text{FGen}, \text{VEval}, \text{Verify})$ for function family $\mathcal{F}$.

interaction between the evaluators [12]. Indeed, we observe that, checks for well-formedness and the auxiliary output is independent of the real point function $f_{\alpha, \beta}$. Thus, the dealer can generate and distribute a pair of VDPF keys $K^{(0)}, K^{(1)}$ for a random special input α' , without the output correction word for the special output β in the preprocessing phase; while the evaluators jointly check the valid of $K^{(0)}, K^{(1)}$. In this online phase, the dealer can broadcasts $\delta_\alpha := \alpha - \alpha'$ and the output correction word cw for β to make VDPF keys complete.

Next, we utilize Schnorr's protocol to prove that D knows the discrete logarithm sk of the select $\text{vk} := \Lambda_\alpha$. Obviously, the commitment and challenge steps of Schnorr's protocol is independent of the statement vk and the witness sk and, thus they can be performed in the preprocessing phase. That is, D pre-broadcasts $t := g^v$ and then the evaluators P_0, P_1 sends a random $c \in \mathbb{Z}_{p-1}$ to D. The challenge is the contradiction between the privacy requirement that each P_b should not know the statement vk and the purpose that the evaluators can locally make a decision without communication to each other. Inspired by [37], we let the dealer commit to the statement (i.e., the secret shares of vk from the inner product between Λ and the evaluation results of VDPF keys) to the evaluators, then prove that the rest steps of Schnorr's protocol over vk and sk are processed in a proper way. In particular, these commitments are built by multi-verifier VOLE correlations, which is adopted from [37]. In Fig. 12, we recap the functionality of multi-verifier subfield VOLE of [37] in our setting⁷, and reduce it to a 2-verifier VOLE functionality $\mathcal{F}_{\text{mv-VOLE}}^p$. According to Fig. 12, in the preprocessing phase, each P_b gets a global key $\Delta^{(b)}$ from the multi-verifier VOLE functionality $\mathcal{F}_{\text{mv-VOLE}}^p$. For any vector $\mathbf{w} \in \mathbb{Z}_p^3$, D gets MAC tags $\mathbf{m}^{(0)}, \mathbf{m}^{(1)} \in \mathbb{Z}_p^3$ and the vector $\mathbf{w}$ from $\mathcal{F}_{\text{mv-VOLE}}^p$; while each P_b gets the private key $\mathbf{m}^{(b)} \in \mathbb{Z}_p^3$, such that $\forall i \in \mathbb{Z}_3$, $k_i^{(b)} = m_i^{(b)} + w_i \cdot \Delta^{(b)}$.

In the online phase, the dealer first computes $\text{vk}^{(0)}, \text{vk}^{(1)}$ using the VDPF keys it distributes and commits to $(\text{vk}^{(0)})^c$ and $(\text{vk}^{(1)})^c$ by broadcasting $\delta_0 := (\text{vk}^{(0)})^c - w_0$ and $\delta_1 := (\text{vk}^{(1)})^c - w_1$ to all evaluators. Then the dealer commits to $\text{vk}^c := (\text{vk}^{(0)})^c \cdot (\text{vk}^{(1)})^c$ by broadcasting $\delta_2 := \text{vk}^c - w_2$. Each evaluator can verify these commitments after receiving the corresponding MAC tags from D privately. Finally, the dealer broadcasts $r := v - c \cdot \text{sk}$ and the evaluators locally check if $g^r \cdot \text{vk}^c = t$ using commitments to make a decision.

Security. We provide the intuition for the security of our $\Pi_{\text{ni-CDPF}}$ in Fig. 11. First, the completeness clearly

⁷Recall that, we consider one dealer and two verifiers, and the malicious security for a single corrupted party.

Protocol $\Pi_{\text{ni-CDPF}}$

Parameters: A prime number p , a group $\mathbb{G} := \mathbb{Z}_p^*$ with a generator g , a family $\mathcal{F} := \{f : \{0, 1\}^n \rightarrow \tilde{\mathbb{G}}\}$ of point functions, an integer $N := 2^n$, a hash function $H : \mathbb{Z}_p \rightarrow \{0, 1\}^\lambda$ sampled from a collision resistant family.

Inputs: The dealer D and evaluators P_0, P_1 hold constraint configuration list $\Lambda \in \mathbb{G}^N$. The dealer D additionally holds the point function $f_{\alpha, \beta}$ and secret key sk .

Preprocessing Phase: The point function $f_{\alpha, \beta}$ and secret key sk are unknown by D .

1. D generates and distributes incomplete VDPF keys, and P_0, P_1 check if these keys are valid.
 - (a) D picks random $\alpha' \leftarrow \mathbb{Z}_N$, generates a pair of VDPF keys for the point function $f_{\alpha', (\beta, 1)}^{(0)}$, except for the correction word for the unknown β . Denote these incomplete VDPF keys as $K^{(0)}, K^{(1)}$. D distributes $K^{(0)}, K^{(1)}$ to P_0, P_1 .
 - (b) Each P_b evaluates $K^{(b)}$ on the input set $\mathbb{Z}_N$, except for the first output item correction, to obtain the VDPF audit token $\tau^{(b)}$ and the secret share $\{y_i^{(b)}\}_{i \in \mathbb{Z}_N}$ of the second output item, and computes $\delta^{(b)} := H(b + (-1)^b \cdot \sum_{i \in \mathbb{Z}_N} y_i^{(b)})$.
 - (c) P_0 and P_1 exchange $\tau^{(0)}, \tau^{(1)}$ and $\delta^{(0)}, \delta^{(1)}$, and check if $\text{VDPF.Verify}(\tau^{(0)}, \tau^{(1)}) = 1$ and $\delta^{(0)} = \delta^{(1)}$.
2. D picks random $v \leftarrow \mathbb{Z}_{p-1}$ and broadcasts $t := g^v \in \mathbb{G}$. Then, P_0 and P_1 agree on a random $c \in \mathbb{Z}_{p-1}$ and send c to D .
3. D and P_0, P_1 jointly generates VOLE correlations.
 - (a) D and P_0, P_1 send $(\text{init}, \text{sid})$ to $\mathcal{F}_{\text{mv-VOLE}}^p$. Then $\mathcal{F}_{\text{mv-VOLE}}^p$ returns $\Delta^{(b)}$ to each P_b .
 - (b) D picks random $w_0, w_1, w_2 \leftarrow \mathbb{Z}_p$ and sets $\mathbf{w} := (w_0, w_1, w_2)$;
 - (c) D sends $(\text{extend}, \text{sid}, 3, \mathbf{w})$ to the functionality $\mathcal{F}_{\text{mv-VOLE}}^p$ while P_0, P_1 sends $(\text{extend}, \text{sid}, 3)$ to $\mathcal{F}_{\text{mv-VOLE}}^p$. Then $\mathcal{F}_{\text{mv-VOLE}}^p$ returns $(\mathbf{m}^{(0)}, \mathbf{m}^{(1)})$ to D and returns $\mathbf{k}^{(b)}$ to each P_b , such that $\forall i \in \mathbb{Z}_3, k_i^{(b)} = m_i^{(b)} + w_i \cdot \Delta^{(b)}$.

Online Phase: D knows the point function $f_{\alpha, \beta}$ and secret key sk

1. D computes and broadcasts $\delta_\alpha := \alpha - \alpha' \in \mathbb{Z}_N$ and the correction word cw for β corresponding the incomplete keys $K^{(0)}, K^{(1)}$. All parties locally combine δ_α, cw and the incomplete keys to obtain the VDPF keys $f_{\alpha, (\beta, 1)}^{(0)}$ and/or $f_{\alpha, (\beta, 1)}^{(1)}$.
2. D and P_0, P_1 compute the correlations of the target verification key:
 - (a) For each $b \in \{0, 1\}$: D computes $(\{(\beta_i, y_i)\}_{i \in \mathbb{Z}_N}, \tau^{(b)}) := \text{VDPF.Eval}(b, f_{\alpha, (\beta, 1)}^{(b)}, \mathbb{Z}_N)$, $\text{vk}^{(b)} := \prod_{i \in \mathbb{Z}_N} (\Lambda_i)^{y_i}$, $m_{\text{vk}, b}^{(0)} := m_b^{(0)}$, $m_{\text{vk}, b}^{(1)} := m_b^{(1)}$, and $w_{\text{vk}, b} := (\text{vk}^{(b)})^c$. Then D broadcasts $\delta_b := w_{\text{vk}, b} - w_b$.
 - (b) Each P_b locally computes $k_{\text{vk}, 0}^{(b)} := k_0^{(b)} + \delta_0 \cdot \Delta^{(b)}$ and $k_{\text{vk}, 1}^{(b)} := k_1^{(b)} + \delta_1 \cdot \Delta^{(b)}$.
3. P_0 and P_1 check whether these correlations are valid:
 - (a) For each $b \in \{0, 1\}$: D sends $m_{\text{vk}, b}^{(b)}$ to P_b .
 - (b) Each P_b evaluates $(\{(\beta_i, y_i)\}_{i \in \mathbb{Z}_N}, \tau^{(b)}) := \text{VDPF.Eval}(b, f_{\alpha, (\beta, 1)}^{(b)}, \mathbb{Z}_N)$, computes $\text{vk}^{(b)} := \prod_{i \in \mathbb{Z}_N} (\Lambda_i)^{y_i}$, and checks if $k_{\text{vk}, b}^{(b)} = m_{\text{vk}, b}^{(b)} + (\text{vk}^{(b)})^c \cdot \Delta^{(b)}$.
4. P_0 and P_1 verify the secret key of D :
 - (a) D computes $w_{\text{vk}} := (\Lambda_\alpha)^c$, $m_{\text{vk}}^{(0)} := m_2^{(0)}$ and $m_{\text{vk}}^{(1)} := m_2^{(1)}$. D broadcasts $\delta_2 := w_{\text{vk}} - w_2$ and $r := v - c \cdot \text{sk}$ while for each $b \in \{0, 1\}$: D sends $m_t^{(b)} := g^r \cdot m_{\text{vk}}^{(b)}$ to P_b .
 - (b) Each P_b computes $k_{\text{vk}}^{(b)} := k_2^{(b)} + \delta_2 \cdot \Delta^{(b)}$, $k_t := g^r \cdot k_{\text{vk}}^{(b)}$ and checks if $k_t^{(b)} = m_t^{(b)} + t \cdot \Delta^{(b)}$.
5. P_0 and P_1 verify the correlation of vk is computed correctly:
 - (a) For each $b \in \{0, 1\}$: D sends $A_0^{(b)} := m_{\text{vk}, 0}^{(b)} \cdot m_{\text{vk}, 1}^{(b)}$ and $A_1^{(b)} := m_{\text{vk}, 0}^{(b)} \cdot w_{\text{vk}, 1} + m_{\text{vk}, 1}^{(b)} \cdot w_{\text{vk}, 0} - m_{\text{vk}}^{(b)}$ to P_b .
 - (b) Each P_b locally computes $B^{(b)} := k_{\text{vk}, 0}^{(b)} \cdot k_{\text{vk}, 1}^{(b)} - k_{\text{vk}}^{(b)} \cdot \Delta^{(b)}$ and checks if $B^{(b)} = A_0^{(b)} + A_1^{(b)} \cdot \Delta^{(b)}$.

Figure 11: Protocol for CDPF with non-interactive evaluators in the $\mathcal{F}_{\text{mv-VOLE}}^p$ -hybrid model

holds, except for the verification of vk^c (step 5 in the online phase). We note that, for each $b \in \{0, 1\}$,

$$\begin{aligned}
 B^{(b)} &:= k_{\text{vk}, 0}^{(b)} \cdot k_{\text{vk}, 1}^{(b)} - k_{\text{vk}}^{(b)} \cdot \Delta^{(b)} \\
 &= (m_{\text{vk}, 0}^{(b)} + (w_{\text{vk}, 0} \cdot \Delta^{(b)}) \cdot (m_{\text{vk}, 1}^{(b)} + w_{\text{vk}, 1} \cdot \Delta^{(b)}) \\
 &\quad - (m_{\text{vk}}^{(b)} + w_{\text{vk}} \cdot \Delta^{(b)}) \cdot \Delta^{(b)} \\
 &= A_0^{(b)} + A_1^{(b)} \cdot \Delta^{(b)} + (w_{\text{vk}, 0} \cdot w_{\text{vk}, 1} - w_{\text{vk}}) \cdot (\Delta^{(b)})^2
 \end{aligned}$$

where $w_{\text{vk}, 0} := (\text{vk}^{(0)})^c$, $w_{\text{vk}, 1} := (\text{vk}^{(1)})^c$ and $w_{\text{vk}} := \text{vk}^c$ if D is honest. Therefore, we have $B^{(b)} = A_0^{(b)} +$

$$A_1^{(b)} \cdot \Delta^{(b)}.$$

For privacy, the preprocessing phase is independent of private input. In the online phase, the dealer broadcasts (1) δ_α , cw for VDPF keys, (2) $\delta_0, \delta_1, \delta_2$ for commitment, and (3) r for Schnorr's proof. It is easy to see, these values are uniformly distributed. In addition, the dealer sends (1) $m_{\text{vk},b}^{(b)}$, (2) $m_t^{(b)}$ and (3) $A_0^{(b)}$, $A_1^{(b)}$ to P_b privately. The first two items open $w_{\text{vk},b} := (\text{vk}^{(b)})^c$ and $w_t := t$ to P_b , which are already known by P_b from the VDPF key and Schnorr's proof. The items in (3) are masked MAC tags and also reveal nothing of dealer's private input to P_b .

The soundness of our $\Pi_{\text{ni-CDPF}}$ has a subtle drawback. Assume the discrete logarithm assumption holds in $\mathbb{G}$. A malicious dealer $\mathcal{A}$ can cheat one of the evaluators, say P_0 , by broadcasting $\delta_1^* := (g^{\text{sk}^*}/\text{vk}^{(0)})^c - w_b$ for a forged secret key sk^* . Then P_0 would accept the forged proof from $\mathcal{A}$. Nevertheless, the other evaluator P_1 can distinguish the invalid δ_1^* . We conclude that any malicious dealer cannot cheat two evaluators at the same time unless it breaks the discrete logarithm assumption.

Functionality $\mathcal{F}_{\text{mv-VOLE}}^p$

The functionality interacts with parties D , P_0 and P_1 as well as an adversary $\mathcal{S}$. It is parameterized with a finite field $\mathbb{F}_p$.

Initialize: Upon receiving $(\text{init}, \text{sid})$ from D , P_0 and P_1 , for each $b \in \{0, 1\}$, if P_b is honest, then sample $\Delta^{(b)} \leftarrow \mathbb{F}_p$; else receive $\Delta^{(b)} \in \mathbb{F}_p$ from $\mathcal{S}$. Store $\Delta^{(b)}$ and send $\Delta^{(b)}$ to P_b . Ignore all subsequent **init** commands.

Extend: Upon receiving $(\text{extend}, \text{sid}, \ell, \mathbf{w})$ from D , and $(\text{extend}, \text{sid}, \ell)$ from P_0 and P_1 , where $\mathbf{w} \in \mathbb{F}_p^\ell$:

- For $b \in \{0, 1\}$: sample $\mathbf{k}^{(b)} \leftarrow \mathbb{F}_p^\ell$, and set $\mathbf{m}^{(b)} := \mathbf{k}^{(b)} - \mathbf{w} \cdot \Delta^{(b)} \in \mathbb{F}_p^\ell$.
- If D is corrupted, for each $b \in \{0, 1\}$: receive $\mathbf{m}^{(b)} \in \mathbb{F}_p^\ell$ from $\mathcal{S}$ and set $\mathbf{k}^{(b)} := \mathbf{m}^{(b)} + \mathbf{w} \cdot \Delta \in \mathbb{F}_p^\ell$.
- For each $b \in \{0, 1\}$: if P_b is corrupted, receive $\mathbf{k}^{(b)} \in \mathbb{F}_p^\ell$ from $\mathcal{S}$ and set $\mathbf{m}^{(b)} := \mathbf{k}^{(b)} - \mathbf{w} \cdot \Delta \in \mathbb{F}_p^\ell$.
- Output $(\mathbf{m}^{(0)}, \mathbf{m}^{(1)})$ to D and $\mathbf{k}^{(b)}$ to each P_b .

Figure 12: Functionality for multi-verifier VOLE [37]

E Security Proofs

E.1 Proof of Theorem 1

Proof. We prove the IVDPF construction in Fig. 4 satisfies the properties in Definition 2.

Correctness. Except for packing the tokens, our batch prefix evaluation VIDPF follows the construction of [27]. Thus, the correctness of function output is trivial. For the batch verification, since each pair of original tokens is identical, as noted in [27], they remain equal after we XOR accumulate all the tokens, i.e., $\text{BVerify}(\tau^{(0)}, \tau^{(1)}) = 1$ as required.

Privacy. We construct an efficient simulator Sim for the view of any evaluator P_b . On input $(1^\lambda, b, n, \{\mathbb{G}_i\}_{i \in [n]}, X)$, the simulator Sim proceeds as follows:

- 1 $s_0^{(b)} \leftarrow \{0, 1\}^\lambda$
- 2 **for** $i := 1$ to n :
- 3 $\text{cw}_i \leftarrow \{0, 1\}^{\lambda+2}$, $\text{cs}_i \leftarrow \{0, 1\}^{4\lambda}$, $\text{ocw}_i \leftarrow \mathbb{G}_i$
- 4 $f^* := (s_0^{(b)}, (\text{cw}_i, \text{cs}_i, \text{ocw}_i)_{i \in [n]})$
- 5 $(\{y_{i,x}^*\}_{i \in [n], x \in X_{[1,i]}}) \leftarrow \text{IVDPF.VEval}(b, f^*, X)$
- 6 **Output** (f^*, τ^*)

In the real view of P_b , the VIDPF key $f^{(b)}$ consists of (1) a random seed $s_0^{(b)}$ which distributed identically to the output of Sim , (2) the correction words $(\text{cw}_i, \text{cs}_i)_{i \in [n]}$, where each one is XOR'd with the output of the PRG G and (3) the output correction words $(\text{ocw}_i)_{i \in [n]}$, where each one is added with the pseudorandom output of G'_i . Since P_b does not know the seeds of G and G'_i , (2) and (3) are computationally indistinguishable from random elements. In addition, the audit token $\tau^{(1-b)}$ is equal to $\tau^{(b)}$, and the evaluator holding $f^{(b)}$ can locally compute it. Therefore, we conclude that the output of Sim is computationally indistinguishable from the view of verifier P_b .

Soundness. Given the input set $X \subseteq \{0, 1\}^n$, each evaluator has $(\{y_{i,x}^{(b)}\}_{i \in [n], x \in X_{1,i}}, \tau^{(b)}) \leftarrow \text{Eval}(b, f_{\alpha,\beta}^{(b)}, X)$. In addition, at every level i , each evaluator P_b has a set of seeds and control bits, denoted as $\{\tilde{s}_{i,x}^{(b)}, t_{i,x}^{(b)}\}_{x \in X_{[1,i]}}$ and output a statement for next evaluation, denoted as $\{\text{st}_{i,x}^{(b)}\}_{x \in X_{[1,i]}}$. The evaluators have the same correction seed cs_i . Let $\tilde{\tau}_{i,x}^{(b)} := H(\tilde{s}_{i,x}^{(b)} || t_{i,x}^{(b)} || x)$ and $\tau_{i,x}^{(b)} := \tilde{\tau}_{i,x} \oplus t_{i,x}^{(b)} \cdot \text{cs}_i$. We consider two cases as follows:

Case 1: At most one non-zero node at each level i . We prove that no PPT adversary $\mathcal{A}$ can construct VIDPF keys such that $\text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1$ when $|\{x \in X_{[1,i]} | y_{i,x}^{(0)} + y_{i,x}^{(1)} \neq 0\}| > 1$ holds for an $i \in [n]$.

Suppose for contradiction that $\mathcal{A}$ constructs a key that satisfies the above condition. There exists $i \in [n]$, two distinct $u, v \in X_{[1,i]}$ such that $\tilde{s}_{i,u}^{(0)} \neq \tilde{s}_{i,u}^{(1)}$ and $\tilde{s}_{i,v}^{(0)} \neq \tilde{s}_{i,v}^{(1)}$, and $\forall x \in X_{[1,i]}$ we have $\tau_{i,x}^{(0)} = \tau_{i,x}^{(1)}$ to make $\text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1$ (due to the collision resistance of H'). Because of the collision resistance of H , we have $\tilde{\tau}_{i,u}^{(0)} || t_{i,u}^{(0)} \neq \tilde{\tau}_{i,u}^{(1)} || t_{i,u}^{(1)}$ and $\tilde{\tau}_{i,v}^{(0)} || t_{i,v}^{(0)} \neq \tilde{\tau}_{i,v}^{(1)} || t_{i,v}^{(1)}$. Hence, the adversary $\mathcal{A}$ needs to find cs_i such that $\text{cs}_i = \tilde{\tau}_{i,u}^{(0)} \oplus \tilde{\tau}_{i,u}^{(1)} = \tilde{\tau}_{i,v}^{(0)} \oplus \tilde{\tau}_{i,v}^{(1)} \neq 0$. Due to the XOR-collision resistance of H , to get this equality we must have one of the following two events: (1) $\tilde{\tau}_{i,u}^{(0)} = \tilde{\tau}_{i,u}^{(1)}$ and $\tilde{\tau}_{i,v}^{(1)} = \tilde{\tau}_{i,v}^{(0)}$; (2) $\tilde{\tau}_{i,u}^{(1)} = \tilde{\tau}_{i,u}^{(0)}$ and $\tilde{\tau}_{i,v}^{(0)} = \tilde{\tau}_{i,v}^{(1)}$. Since $u \neq v$, any equality above violates the collision resistance of H .

Case 2: Non-zero node at level $i+1$ is a child of non-zero node at level i . We prove that no PPT adversary $\mathcal{A}$ can construct VIDPF keys such that $\text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1$ when the non-zero node $u \in X_{[1,i]}$ at level i is not the parent of the non-zero node $v \in X_{[1,i+1]}$ at level $i+1$. Suppose for contradiction that $\mathcal{A}$ constructs a key that satisfies the above condition. There exists a level $i \in [n]$ and two nodes $u \in X_{[1,i]}$ and $v \in X_{[1,i+1]}$, such that $\tilde{s}_{i,u}^{(0)} || t_{i,u}^{(0)} \neq \tilde{s}_{i,u}^{(1)} || t_{i,u}^{(1)}$, $\tilde{s}_{i,v}^{(0)} || t_{i,v}^{(0)} \neq \tilde{s}_{i,v}^{(1)} || t_{i,v}^{(1)}$, $u \neq v_{[1,i]}$, and we have $\tau_{i,k}^{(0)} = \tau_{i,k}^{(1)}$ for $k \in X_{[1,i]}$ and $\tau_{i+1,z}^{(0)} = \tau_{i+1,z}^{(1)}$ for $z \in X_{[1,i+1]}$ to make $\text{Verify}(\tau^{(0)}, \tau^{(1)}) = 1$ (due to the collision resistance of H'). We denote $k := v_{[1,i]}$ and analyze the statements $\text{st}_{i,k}^{(0)}, \text{st}_{i,k}^{(1)}$ for the evaluation on prefix v in the following two events: (1) $\text{st}_{i,k}^{(0)} = \text{st}_{i,k}^{(1)}$. Due to the deterministic PRG G , we have $\tilde{s}_{i,v}^{(0)} || t_{i,v}^{(0)} = \tilde{s}_{i,v}^{(1)} || t_{i,v}^{(1)}$ in this event. So, the node v cannot be non-zero. (2) $\text{st}_{i,k}^{(0)} \neq \text{st}_{i,k}^{(1)}$. Parse the statement $\text{st}_{i,k}^{(b)}$ into $s_{i,k}^{(b)}$ and $t_{i,k}^{(b)}$. The first item $s_{i,k}^{(b)}$ is generated from PRG evaluation $G'_i(\tilde{s}_{i,k}^{(b)})$. Since level i at most a non-zero node proved before, which means at most one value $x \in X_{[1,i]}$ such that $\tilde{s}_{i,x}^{(0)} || t_{i,x}^{(0)} \neq \tilde{s}_{i,x}^{(1)} || t_{i,x}^{(1)}$, we have $u = x = k$. It contracts the condition $u \neq k$.

In conclusion, the construction in Fig. 4 is secure. $\square$

E.2 Proof of Theorem 2

Proof. We prove the IVDPF-CDPF construction in Fig. 5 satisfies the properties in Definition 1.

Correctness We show the correctness of VIDPF-CDPF in Fig. 5. The first condition clearly holds. Next, we prove the second condition.

Due to the correctness of IVDPF and $\alpha \in X$, given a secret-shared all-prefix point function $f_{\alpha,1||\beta}$, for $i := n$, we have $y_{i,x}^{(0)} + y_{i,x}^{(1)} = 1||\beta$ if $x = \alpha$, and 0 otherwise; for $i \in [1, n-1]$, we have $y_{i,x}^{(0)} + y_{i,x}^{(1)} = 1$ if $x = \alpha_{[1,i]}$, and 0 otherwise. As shown in Fig. 5, we parse $(q_{i,x}^{(b)}, y_x^{(b)}) := y_{i,x}^{(b)}$ for $i := n$; and we set $q_{i,x}^{(b)} := y_{i,x}^{(b)}$ for $i \in [1, n-1]$. Thus, for $x \in X$, $y_x^{(0)} + y_x^{(1)} = \beta$ if $x = \alpha$ and $y_x^{(0)} + y_x^{(1)} = 0$ otherwise, i.e., the shares of function outputs $y_x^{(b)}$ are correct.

On the other hand, we show that for the secret key $\text{sk} \leftarrow \text{SKGen}(1^\lambda, \text{id}, \text{msk})$ where $\{\text{id}\} \in \mathbb{A}_\alpha$, the algorithm Verify outputs 1. As described in Fig. 5, the algorithm Verify outputs 1 iff $\text{IVDPF.Verify}(\tau_0^{(0)}, \tau_0^{(1)}) =$

1, $\tau_1^{(0)} = \tau_1^{(1)}, \tau_2^{(0)} = \tau_2^{(1)}, \tau_3^{(0)} = \tau_3^{(1)}$ and $\tau_4^{(0)} = \tau_4^{(1)}$. First, due to $u = u$, it is clear that we have $\tau_4^{(0)} = \tau_4^{(1)}$. The equality of $\text{IVDPF.Verify}(\tau_0^{(0)}, \tau_0^{(1)}) = 1$ follows from the correctness of IVDPF. For the equality of $\tau_1^{(0)} = \tau_1^{(1)}$, it is true if for each $i \in [n]$, $z_{i,0} + z_{i,1} = 1$. As mentioned before, for $i \in [n]$, we have $q_{i,x} := q_{i,x}^{(0)} + q_{i,x}^{(1)} = 1$ if $x = \alpha_{[1,i]}$ and $q_{i,x} = 0$ otherwise. As such, we get $z_{i,\alpha_i} = 1$ and $z_{i,1-\alpha_i} = 0$ so that $z_{i,0} + z_{i,1} = 1$. Finally, to see why it holds that $\tau_2^{(0)} = \tau_2^{(1)}$ and $\tau_3^{(0)} = \tau_3^{(1)}$, observe that $\text{vk} := \text{vk}^{(0)} \cdot \text{vk}^{(1)} = \prod_{i \in [n]} e(g, g)^{r_{i,\alpha_i}} = e(g, g)^{\sum_{i \in [n]} r_{i,\alpha_i}}$. In addition, we have $\text{sk}_\alpha = \{\text{sk}_0, \text{sk}_1, \text{sk}_2\} \in \text{sk}$, $e(u, v) = e(\text{sk}_0, \text{sk}_1) = \sum_{i \in [n]} r_{i,\alpha_i} = \text{vk}$, and $e(u, w) = e(\text{sk}_0, \text{sk}_2) = h$. Thus, it holds that $\frac{e(u, v^{(0)})}{\text{vk}^{(0)}} = \frac{\text{vk}^{(1)}}{e(u, v^{(0)})}$, i.e., $\tau_2^{(0)} = \tau_2^{(1)}$, as required. Similarly, it holds that $e(u, w^{(0)}) = \frac{h}{e(u, w^{(0)})}$, i.e., $\tau_3^{(0)} = \tau_3^{(1)}$.

Privacy To prove the privacy of VIDPF-CDPF in Fig. 5, we construct an efficient simulator Sim for the view of any evaluator P_b . Since IVDPF is secure, there exists a PPT simulator $\text{Sim}_{\text{IVDPF}}$ who can generate a computationally indistinguishable view for IVDPF. Our simulator Sim black-box uses such $\text{Sim}_{\text{IVDPF}}$. On input $(1^\lambda, b, \Lambda, X)$, Sim proceeds as follows:

- 1 $u, v^{(b)}, w^{(b)} \leftarrow \mathbb{G}$;
- 2 $\pi^{(b)} := (u, v^{(b)}, w^{(b)})$
- 3 $(f^{(b)}, \tau_0^{(1-b)}) \leftarrow \text{Sim}_{\text{IVDPF}}(1^\lambda, b, \mathcal{F}, \mathbb{Z}_N)$
- 4 Compute $(Y, \tau^{(b)} := (\tau_0^{(b)}, \tau_1^{(b)}, \tau_2^{(b)}, \tau_3^{(b)}, \tau_4^{(b)})) \leftarrow \text{VEval}(b, \Lambda, f^{(b)}, \pi^{(b)}, X)$
- 5 $\tau_1^{(1-b)} := \tau_1^{(b)}, \tau_2^{(1-b)} := \tau_2^{(b)}, \tau_3^{(1-b)} := \tau_3^{(b)}, \tau_4^{(1-b)} := \tau_4^{(b)}$
- 6 $\tau^{(1-b)} := (\tau_0^{(1-b)}, \tau_1^{(1-b)}, \tau_2^{(1-b)}, \tau_3^{(1-b)}, \tau_4^{(1-b)})$
- 7 Output $(f^{(b)}, \pi^{(b)}, \tau^{(1-b)})$

Below, we prove the indistinguishability through a series of hybrid worlds.

Hyb₀: It is the real view of the evaluator P_b .

Hyb₁: It is same as **Hyb₀**, except that in **Hyb₁** we set the tokens $\tau_1^{(1-b)} := \tau_1^{(b)}, \tau_2^{(1-b)} := \tau_2^{(b)}, \tau_3^{(1-b)} := \tau_3^{(b)}, \tau_4^{(1-b)} := \tau_4^{(b)}$ where $(Y, \tau^{(b)} := (\tau_0^{(b)}, \tau_1^{(b)}, \tau_2^{(b)}, \tau_3^{(b)}, \tau_4^{(b)})) \leftarrow \text{VEval}(b, \Lambda, f^{(b)}, \pi^{(b)}, X)$, instead of computing them from $f^{(1-b)}, \pi^{(1-b)}$.

Claim 1. *If the CDPF scheme has correctness, then **Hyb₁** and **Hyb₀** are perfectly indistinguishable.*

Proof. As described in Fig. 5, **Verify** outputs 1 only if $\tau_1^{(0)} = \tau_1^{(1)}, \tau_2^{(0)} = \tau_2^{(1)}, \tau_3^{(0)} = \tau_3^{(1)}, \tau_4^{(0)} = \tau_4^{(1)}$. Therefore, the tokens $\tau_1^{(1-b)}, \tau_2^{(1-b)}, \tau_3^{(1-b)}, \tau_4^{(1-b)}$ in **Hyb₁** are the same as in **Hyb₀**. So **Hyb₁** and **Hyb₀** are perfectly indistinguishable. $\square$

Hyb₂: It is same as **Hyb₁**, except that in **Hyb₂**, all the items $u, v^{(b)}, w^{(b)}$ in proof $\pi^{(b)}$ are uniformly random.

Claim 2. ***Hyb₂** and **Hyb₁** are perfectly indistinguishable.*

Proof. In both **Hyb₁** and **Hyb₂**, values $u, v^{(b)}$ and $w^{(b)}$ are distributed uniform. Hence, **Hyb₂** and **Hyb₁** are perfectly indistinguishable. $\square$

Hyb₃: It is same as **Hyb₂**, except that in **Hyb₂** the function share $f^{(b)}$ and the token $\tau_0^{(1-b)}$ are generated by $\text{Sim}_{\text{IVDPF}}$.

Claim 3. *If IVDPF is a secure VDPF, then **Hyb₃** and **Hyb₂** are computationally indistinguishable.*

Proof. $(f^{(b)}, \tau_0^{(1-b)})$ is the view of a VIDPF evaluator. Due to the privacy of VIDPF, the output of $\text{Sim}_{\text{IVDPF}}$ is computationally indistinguishable of the real view of any one VIDPF evaluator. Therefore, **Hyb₃** and **Hyb₂** are computationally indistinguishable. $\square$

P_b 's view in **Hyb₃** is identical to the simulated view. In conclusion, the distribution of the output of Sim is computationally indistinguishable from the distribution of the real view of P_b .

Soundness We prove that the VIDPF-CDPF in Fig. 5 is sound. This will be done via a sequence of hybrids.

Hyb₀: It is the soundness game $\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}(\lambda)$ as shown in Fig. 2, where CDPF is the VIDPF-CDPF scheme defined in Fig. 5. Let S_0 be the event that **Hyb₀** outputs 1.

Hyb₁: It is the same as **Hyb₀**, except that the hash functions $H, H', \tilde{H}$ in **VEval** are replaced by the identity function $I(x) := x$. Let S_1 be the event that **Hyb₁** outputs 1.

Claim 4. *If $H, H', \tilde{H}$ are collision-resistant hash functions, then*

$$|\Pr[S_0] - \Pr[S_1]| \leq \text{negl}(\lambda)$$

Proof. We have changed the following six hash values into their pre-images. For readability, we denote by u_b the value of u received by P_b . $\tau_1^{(0)} := H'(\delta_1^{(0)} || \dots || \delta_n^{(0)})$, $\tau_1^{(1)} := H'(\delta_1^{(1)} || \dots || \delta_n^{(1)})$, $\tau_2^{(0)} := H(e(u_0, v^{(0)})/\text{vk}^{(0)})$, $\tau_2^{(1)} := H(\text{vk}^{(1)}/e(u_1, v^{(1)}))$, $\tau_3^{(0)} := H(e(u_0, w^{(0)}))$, $\tau_3^{(1)} := H(h/e(u_1, w^{(1)}))$, $\tau_4^{(0)} := \tilde{H}(u_0)$, $\tau_4^{(1)} := \tilde{H}(u_1)$. They are used in **Verify** to check whether $\tau_1^{(0)} = \tau_1^{(1)}$, $\tau_2^{(0)} = \tau_2^{(1)}$, $\tau_3^{(0)} = \tau_3^{(1)}$ and $\tau_4^{(0)} = \tau_4^{(1)}$. Therefore, for any given input and randomness, the bad event that $S_0 \neq S_1$ happens only if $\delta_1^{(0)} || \dots || \delta_n^{(0)} \neq \delta_1^{(1)} || \dots || \delta_n^{(1)}$ but their hash values are same, or $e(u_0, v^{(0)})/\text{vk}^{(0)} \neq \text{vk}^{(1)}/e(u_1, v^{(1)})$ but their hash values are same, or $e(u_0, w^{(0)}) \neq h/e(u_1, w^{(1)})$, or $u_0 \neq u_1$ but $\tau_4^{(0)} = \tau_4^{(1)}$. Due to the collision-resistance of $H, H', \tilde{H}$, the bad event is bounded by a negligible probability. Hence, $|\Pr[S_0] - \Pr[S_1]| \leq \text{negl}(\lambda)$. $\square$

Hyb₂: It is same as **Hyb₁**, except that we additionally check the function shares $f^{(0)}, f^{(1)}$ outputted by $\mathcal{A}$. More specifically, we use the VIDPF evaluation results $\{y_{i,x}^{(0)}\}_{i \in [n], x \in X_{[1,i]}}$, $\{y_{i,x}^{(1)}\}_{i \in [n], x \in X_{[1,i]}}$ of $f^{(1)}, f^{(0)}$ (that are evaluated in **VEval**) to construct a function $f'(x_1, \dots, x_i) := y_{i,x}^{(0)} + y_{i,x}^{(1)}$ for $i \in [n], x \in X_{[1,i]}$. If there exists a value $\alpha \in \{0, 1\}^n$, such that for $i \in [n], x \in X_{[1,i]}$, $y_{i,x}^{(0)} + y_{i,x}^{(1)} = 0$ when $x \neq \alpha_{[1,i]}$, then continue; else, the game outputs 0 directly. Let S_2 be the event that **Hyb₂** outputs 1.

Claim 5. *If IVDPF is a secure VIDPF scheme, then*

$$|\Pr[S_1] - \Pr[S_2]| \leq \text{negl}(\lambda)$$

Proof. Both in **Hyb₁** and **Hyb₂**, **Verify** checks the VIDPF tokens of $f^{(0)}$ and $f^{(1)}$ evaluation results as $\text{IVDPF.BVerify}(\tau_0^{(0)}, \tau_0^{(1)})$. The bad event that $S_1 \neq S_2$ happens only if $\text{IVDPF.BVerify}(\tau_0^{(0)}, \tau_0^{(1)}) = 1$ but f' is not valid. Due to the soundness of IVDPF, the bad event is bounded by a negligible probability. Hence, we have $|\Pr[S_1] - \Pr[S_2]| \leq \text{negl}(\lambda)$. $\square$

Hyb₃: It is same as **Hyb₂**, except that we replace the oracle **GetKey** by a new oracle **GetKey'** as follows: for each query, **GetKey'** does

- 1 $r \leftarrow \mathbb{Z}_p^*$; $\text{sk}_{i,0} := g^r$;
- 2 $\text{sk}_{i,2} := g^{s/r}$;
- 3 $k := (\text{sk}_{i,0}, \text{sk}_{i,2})$;
- 4 Respond k to $\mathcal{A}$.

Let S_3 be the event that **Hyb₃** outputs 1.

Claim 6. *If the CDH assumption holds in $\mathbb{G}$, then $|\Pr[S_2] - \Pr[S_3]| \leq \text{negl}(\lambda)$*

Proof. Denote $d_i := \sum_{j \in [n]} r_{j,i_j}$. The bad event that $S_2 \neq S_3$ occurs only if the adversary computes valid $\text{sk}_{\alpha,0}, \text{sk}_{\alpha,1}$ such that $e(\text{sk}_{\alpha,0}, \text{sk}_{\alpha,1}) = e(g, g)^{d_\alpha}$, with the help of $\{\text{sk}_{i,1}\}_{i \neq \alpha}$. First, due to the random $\text{sk}_{i,0}$, the group element $\text{sk}_{i,1}$ is independent with g^{d_α} and $e(g, g)^{d_\alpha}$. Thus, each group element $\text{sk}_{i,1}$ itself provides no additional information about sk_α to the adversary. Second, we consider the combination of secret keys from oracle **GetKey**. For any i , $\mathcal{A}$ cannot compute g^{d_i} or $g^{d_i/s}$ with the help of $\text{sk}_{i,1}$, with more than negligible probability; otherwise, $\mathcal{A}$ can break the CDH assumption in $\mathbb{G}$. So $\mathcal{A}$ cannot extract g^{d_α} or $g^{d_\alpha/s}$ from the combination of secret keys with more than negligible probability. Combining together, we have $|\Pr[S_2] - \Pr[S_3]| \leq \text{negl}(\lambda)$. $\square$

Claim 7. *If the DBDH assumption holds in the bilinear map $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_t$, then $\Pr[S_3] \leq \text{negl}(\lambda)$.*

Proof. Suppose for contradiction that there exists a PPT adversary $\mathcal{A}$ and a non-negligible function p , such that $\Pr[S_3] \geq p$. We construct an efficient adversary $\mathcal{B}$ that breaks the DBDH assumption using $\mathcal{A}$. Let $\widetilde{\text{CDPF}} := (\widetilde{\text{Constraint}}, \widetilde{\text{SKGen}}, \widetilde{\text{FGen}}, \widetilde{\text{VEval}}, \widetilde{\text{Verify}})$ be the modified CDPF described above. On input $(1^\lambda, g^a, g^b, g^c, z)$, $\mathcal{B}$ proceeds as follows:

- 1 Randomly sample $\gamma \leftarrow \{0, 1\}$;
- 2 $\{(\mathcal{Q}_i, \mathbb{A}_i)\}_{i \in \mathbb{Z}_N} \leftarrow \mathcal{A}'(1^\lambda, \mathcal{F})$;
- 3 **if** $\mathcal{Q}_i \neq \{f_{i,v}\}_{v \in \tilde{\mathbb{G}}}$: **return** γ ;
- 4 **for** $j \in [n]$:
- 5 $r_{j,0}, r_{j,1} \leftarrow \mathbb{Z}_p$;
- 6 $k_{j,0} := z^{r_{j,0}}; k_{j,1} := z^{r_{j,1}}$;
- 7 $h := e(g^a, g^b)$;
- 8 $\Lambda := (\{(k_{j,0}, k_{j,1})\}_{j \in [n]}, h)$;
- 9 $((f^{(0)}, \pi^{(0)}), (f^{(1)}, \pi^{(1)}), X) \leftarrow \mathcal{A}^{\text{GetKey}'(\cdot)}(1^\lambda, \Lambda)$;
- 10 Parse $\pi^{(0)} := (u, v^{(0)}, w^{(0)}), \pi^{(1)} := (u, v^{(1)}, w^{(1)})$;
- 11 **for** $b \in \{0, 1\}$:
- 12 Compute $(\{y_x^{(b)}\}_{x \in X}, \tau^{(b)}) \leftarrow \widetilde{\text{VEval}}(b, \Lambda, f^{(b)}, \pi^{(b)}, X)$ and record VIDPF evaluation results $\{y_{i,x}^{(b)}\}_{i \in [n], x \in X_{[1,i]}}$;
- 13 Extract the function $f(x) := \begin{cases} y_x^{(0)} + y_x^{(1)}, & x \in X \\ 0, & \text{otherwise} \end{cases}$
- 14 **if** f has a trivial $\text{sk} := \perp$, i.e. $\text{Predicate}(f, X) = 1$: **output** γ ;
- 15 Search $\alpha \in \{0, 1\}^n$, s.t. $\forall i \in [n], x \in X_{[1,i]}, y_{i,x}^{(0)} + y_{i,x}^{(1)} = 0$ when $x \neq \alpha_{[1,i]}$;
- 16 **if** there is no such α : **output** γ ;
- 17 $d_\alpha := \sum_{j \in [n]} r_{j,\alpha_j}; v := v^{(0)} \cdot v^{(1)}; w := w^{(0)} \cdot w^{(1)}$;
- 18 **Output** $e(v, g) = e(w, (g^c)^{d_\alpha})$.

For each query $\text{GetKey}'(\text{id})$ made by $\mathcal{A}$, $\mathcal{B}$ proceeds as follows:

- 1 $r \leftarrow \mathbb{Z}_p^*$;
- 2 $\text{sk}_{i,0} := (g^a)^r; \text{sk}_{i,2} := (g^b)^{1/r}$;
- 3 **Respond** $k := (\text{sk}_{i,0}, \text{sk}_{i,2})$ to $\mathcal{A}$.

Analyze the behavior of $\mathcal{B}$. The input to $\mathcal{B}$ is generated by uniformly random $a, b, c, x \in \mathbb{Z}_p$ and $z \leftarrow (e(g, g)^{abc}, e(g, g)^x)$. $\mathcal{B}$ randomly samples $r_{j,0}, r_{j,1} \leftarrow \mathbb{Z}_N$ for $j \in [n]$, then uses z and these random values to generate $\{k_{j,0}, k_{j,1}\}$ as verification keys. In addition, $\mathcal{B}$ sets $h := e(g^a, g^b)$. When $\mathcal{A}$ makes GetKey' query, $\mathcal{B}$ picks random r , and generates $k := ((g^a)^r, (g^b)^{1/r})$. Note that, $e((g^a)^r, (g^b)^{1/r}) = e(g^a, g^b) = h$. Clearly, the view of $\mathcal{A}$ when run as a subroutine by $\mathcal{B}$ is perfectly indistinguishable to its view in real game.

We argue that $\mathcal{B}$ wins if S_2 happens. First, if S_2 happens, then (1) there exists a value $\alpha \in \{0, 1\}^n$, such that for $i \in [n], x \in X_{[1,i]}, y_{i,x}^{(0)} + y_{i,x}^{(1)} = 0$ if $x \neq \alpha_{[1,i]}$, (2) $\tau_1^{(0)} = \tau_1^{(1)}$, (3) $\tau_2^{(0)} = \tau_2^{(1)}$, i.e., $e(u, v^{(0)})/\text{vk}^{(0)} = \text{vk}^{(1)}/e(u, v^{(1)})$, and (4) $\tau_3^{(0)} = \tau_3^{(1)}$, i.e., $e(u, w^{(0)}) = h/e(u, w^{(1)})$. Due to (1), (2) and (3), the shares of verification key computed in $\widetilde{\text{VEval}}$ satisfy $\text{vk}^{(0)} \cdot \text{vk}^{(1)} = \text{vk}_\alpha = z^{d_\alpha} = e(u, v)$. Due to (4), it holds that $e(u, w) = h = e(g, g)^{ab}$. Easy to note that, if $z = e(g, g)^{abc}$, it holds that $w^{c \cdot d_\alpha} = v$.

Therefore, we have $\left| \Pr[\mathcal{B}(e, g^a, g^b, g^c, r) = (r = e(g, g)^{abc})] - \frac{1}{2} \right| \geq \Pr[S_3] \geq p$. That is, $\mathcal{B}$ succeeds to distinguish the tuples $(g^a, g^b, g^c, e(g, g)^{abc})$ and $(g^a, g^b, g^c, e(g, g)^z)$ with a non-negligible advantage, contradicting the DBDH assumption. $\square$

Combining together, $\Pr[\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}(\lambda) = 1] \leq \text{negl}(\lambda)$ is proven. $\square$

E.3 Proof of Theorem 3

Proof. We prove the $\star$ -CDPF construction in Fig. 6 satisfies the properties in Definition 1.

Completeness. Due to the completeness of VDPF, for the secret-shared point function $f_{\alpha,\beta}$, we have $\llbracket y_i \rrbracket = \beta$ if $i = \alpha$ and $y_i = 0$ otherwise. For the secret-shared point function $f_{\eta,\beta}$, we have $\llbracket \tilde{y}_i \rrbracket = \beta$ if $j = \eta$ and $\llbracket \tilde{y}_i \rrbracket = (0, 0)$ otherwise. We show that if there exists ρ such that $\beta \wedge \mathbf{rs}_{\alpha,\rho} = 0$ (i.e., $f_{\alpha,\beta} \in \mathcal{Q}_{\alpha,\rho}$), the algorithm Verify outputs 1. In Fig. 6, Verify outputs 1 if and only if $\forall i \in [0, 1], \text{VDPF.Verify}(\tau_i^{(0)}, \tau_i^{(1)}) = 1$ and $\forall i \in [2, 3], \tau_i^{(0)} = \tau_i^{(1)}$. The equality of the former follows from the completeness of VDPF. The equality of $\tau_2^{(0)} = \tau_2^{(1)}$ follows from $\eta \in [\alpha \cdot \ell, \alpha \cdot \ell + \ell)$. For the equality of $\tau_3^{(0)} = \tau_3^{(1)}$, observe that $\mathbf{rs}_{\alpha,\rho} \wedge \bigoplus_{i \in \mathbb{Z}_{N\ell}} \llbracket \tilde{y}_i \rrbracket = \mathbf{rs}_{\alpha,\rho} \wedge \llbracket \beta \rrbracket = \llbracket 0 \rrbracket$. Hence, it holds that $\tau_3^{(0)} = \tau_3^{(1)}$, as required.

Privacy. We construct an efficient simulator Sim for the view of any verifier P_b . We use the efficient simulator Sim_{VDPF} to generate the view of the VDPF output. On input $(1^\lambda, b, \Lambda, X)$, the simulator Sim proceeds as follows:

- 1 $(f^{(b)}, \tau_0^{(1-b)}) \leftarrow \text{Sim}_{\text{VDPF}}(1^\lambda, b, \mathcal{F}, \mathbb{Z}_N)$
- 2 $(\tilde{f}^{(b)}, \tau_1^{(1-b)}) \leftarrow \text{Sim}_{\text{VDPF}}(1^\lambda, b, \mathcal{F}', \mathbb{Z}_{N\ell})$
- 3 Compute $\tau_2^{(b)}, \tau_3^{(b)}$ using $\Lambda, f^{(b)}$ and $\tilde{f}^{(b)}$ according to VEval described in Fig. 6
- 4 $\tau_2^{(1-b)} := \tau_2^{(b)}, \tau_3^{(1-b)} := \tau_3^{(b)}$
- 5 Output $(f^{(b)}, \pi^{(b)} := \tilde{f}^{(b)}, \tau^{(1-b)} := \{\tau_i^{(1-b)}\}_{i \in [0, 3]})$

The output distribution of Sim is computationally indistinguishable from the distribution of the real view of P_b , because: (1) the function share $f^{(b)}$, proof share $\pi^{(b)}$ and audit tokens $\tau_0^{(1-b)}, \tau_1^{(1-b)}$ are output by the simulator Sim_{VDPF} , which guarantees the computational indistinguishability; (2) the audit tokens $\tau_2^{(1-b)}, \tau_3^{(1-b)}$ are equal to $\tau_2^{(b)}, \tau_3^{(b)}$ in the real view due to the CDPF completeness.

Soundness. Suppose for contradiction that there exists a PPT adversary $\mathcal{A}$ and a non-negligible function δ such that

$$\Pr[\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}(\lambda) = 1] \geq \delta(\lambda)$$

Extract $f, \tilde{f}$ from the output by $\mathcal{A}$ in $\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}(\lambda)$. As described in Fig. 6, it holds that $R(\cdot, \cdot) \equiv 1$ in $\star$ -CDPF scheme, which implies $T := \bigcup_{i \in \mathbb{Z}_N} \mathcal{Q}_i$. Hence, to win the game $\mathbf{G}_{\text{CDPF}, \mathcal{A}}^{\text{sound}}$, the adversary $\mathcal{A}$ need to make Verify output 1 and $\forall i, f \notin \mathcal{Q}_i$. In Fig. 6, Verify outputs 1 if and only if (1) $\forall i \in [0, 1], \text{VDPF.Verify}(\tau_i^{(0)}, \tau_i^{(1)}) = 1$, (2) $\tau_2^{(0)} = \tau_2^{(1)}$, and (3) $\tau_3^{(0)} = \tau_3^{(1)}$. For (1), by the soundness property of VDPF, we can assume $f, \tilde{f}$ encodes point functions $f_{\alpha,\beta}, f_{\eta,\beta'}$, respectively. Let $\{y_i\}, \{\tilde{y}_i\}$ be evaluation results of VDPF keys of $f_{\alpha,\beta}, f_{\eta,\beta'}$. For (2), if $\eta \notin [\alpha \cdot \ell, \alpha \cdot \ell + \ell)$ or $\beta \neq \beta'$, which means that $\exists i \in \mathbb{Z}_N$ such that

$$y_i^{(0)} \oplus \left(\bigoplus_{j \in \mathbb{Z}_\ell} \tilde{y}_{i \cdot \ell + j}^{(0)} \right) \neq y_i^{(1)} \oplus \left(\bigoplus_{j \in \mathbb{Z}_\ell} \tilde{y}_{i \cdot \ell + j}^{(1)} \right),$$

the adversary $\mathcal{A}$ should find a string s such that for an adversarially chosen $b \in \{0, 1\}$, for $i \in \mathbb{Z}_N$, $\delta_i := y_i^{(b)} \oplus \left(\bigoplus_{j \in \mathbb{Z}_\ell} \tilde{y}_{i \cdot \ell + j}^{(b)} \right)$, it holds that

$$H(s) = H(\delta_0 || \dots || \delta_{N-1})$$

but $s \neq \delta_0 || \dots || \delta_{N-1}$. It contradicts the collision resistance of H . Hence, we can assume $\eta \in [\alpha \cdot \ell, \alpha \cdot \ell + \ell)$ and $\beta = \beta'$. For (3), let $\rho := \eta - \alpha\ell$. If $\nexists i, f \in \mathcal{Q}_i$, i.e., $\beta \wedge \mathbf{rs}_{\alpha,\rho} \neq 0$, the adversary $\mathcal{A}$ should find a string s' such that for an adversarially chosen $b \in \{0, 1\}$, it holds that

$$H(s') = H\left(\bigoplus_{i \in \mathbb{Z}_N, j \in \mathbb{Z}_\ell} \mathbf{rs}_{i,j} \wedge \tilde{y}_{i \cdot \ell + j}^{(b)} \right)$$

but $s' \neq \bigoplus_{i \in \mathbb{Z}_N, j \in \mathbb{Z}_\ell} \mathbf{rs}_{i,j} \wedge \tilde{y}_{i \cdot \ell + j}^{(b)}$. It contradicts the collision resistance of H . $\square$